

İSTANBUL TEKNİK ÜNİVERSİTESİ



ÖĞRENCİ STAJ RAPORU STUDENT INTERN REPORT

FAKÜLTE *FACULTY*

ELEKTRİK ELEKTRONİK FAKÜLTESİ

BÖLÜM *DEPARTMENT*

ELEKTRİK MÜHENDİSLİĞİ

ÖĞRENCİNİN *STUDENT*

Adı Soyadı Name <i>Surname</i>	Mustafa Emre Çekmiş
Numarası <i>Student Number</i>	040230114
Staj Komisyonu Başkanı <i>Head of Internship Comission</i>	Dr. Öğr. Üyesi Aydın Tarık Zengin

STAJ BİLGİLERİ *INTERNSHIP INFORMATION*

Staj Türü <i>Internship Type</i>	Research and Development
Staj Yeri <i>Internship Location</i>	Youtune Technology Co.
Staj Başlama Tarihi <i>Internship Start Date</i>	06.07.2026
Staj Bitiş Tarihi <i>Internship End Date</i>	18.09.2026
Staj Gün Sayısı <i>Internship Duration (days)</i>	55

İçindekiler

1. Kuruluş Hakkında Bilgiler	3
2. Giriş	4
Orientation and Definition of the Project Scope	5
Setting Up the Local Development Enviroment	7
Running the Local Model as a Background Server	8
First Caching Layer and an Offline Similarity Method	9
Threshold Calibration and First Timing Measurements	10
Attempting to Integrate the GPTCache Library	12
Writing a Custom Offline Cache with Permanent Storage	14
Adding a Real Embedding Model as a Second Local Server	15
Diagnosing Persistent Wrong Cache Hits.....	17
Diagnosing Persistent Wrong Cache Hits (continued).....	19
Building the First Retrieval Pipeline (RAG).....	21
Testing Question Standardisation and Fixing the Entity Check.....	23
Narrowing the System to a Single Subject Area	25
Building a Proactive Cache Warming System	27
Finding and Fixing a Topic Collision in the Cache	29
Finding and Fixing a Topic Collision in the Cache (continued).....	31
Speeding Up Cache Hits and Investigating an Idle Delay.....	33
Finding the Cause of the Idle Delay	34
Rewriting the Similarity Search as a Matrix Operation	36
Expanding the Prepared Question Set to 600 Combinations	37
Translating the Code to English and Preparing Report Evidence	38
Writing and Submitting the Monthly Progress Report	39
Testing Thread Settings to Reduce Cache Warming Time	40
Expanding the Prepared Question Set to 1,000 Combinations	41

Evaluating the Expanded Set and Returning to 600	42
Reviewing the State of the System Before the Next Stage	43
Weekly Meeting and Agreeing the Next Stage	44
Making the Similarity Calculation a True Cosine Similarity	45
Benchmarking Matrix Similarity Against the Loop Method	46
Finding That the Matrix Multiplication Was Not the Bottleneck	47
Adding an In-Memory Matrix Cache	48
Deciding to Move the Project to an Electrical Engineering Field	49
Choosing the Electrical Engineering Area for the Project	50
First Electrical Engineering Document in the Retrieval Pipeline	51
Technical Visit to Minth Group, Ningbo	52
Loading a Real Standard and Fixing Two Retrieval Problem	53
Connecting the Cache to Retrieval and Limiting Runaway Answers	54
Pipeline Verification and System Prompt Correction	55
Building the Measurement Harness	56
Two Cache Bugs Found by Measurement	57
Measuring Whether the Answers Are Correct	58
A Second Document, Source Separation and Diagnostic Tools	59
Writing the August Monthly Progress Report	60
Automated Token Density and Chunk Sizing	61
Folder-Based Loading and a Run That Came Out Worse	62
Investigating the Regression and Designing a Controlled Test	63
Confirming the Chunking Method and Fixing Table Retrieval	64
Measuring the Table-Aware Chunking Change	65
Detecting Changed Documents	66
Rebuilding the Interactive Assistant and Testing It by Hand	67
Explaining the Failure That Has Been Open Since August	68

Freezing the Code and Running the Final Measurement	69
Choosing How to Benchmark the System.....	70
Running the Retrieval Benchmark	71
Checking the Statistics, Then Writing the Final Report	72
Verifying the Report and Delivering It	73
Freez Final Day, Handover and Closing the Internship	74

Uygulamanın Yapıldığı Birim:	-	Tarih: 06.07.2026
Yapılan Uygulama:	1. Kuruluş Hakkında Bilgiler	
I carried out my internship at Youtune Technology Co., Ltd. (浙江禹调科技有限公司), a technology company registered in the People's Republic of China under unified social credit code 91330212MAEJM97K28. The company's office is at Room 701, Building No. 2, Zhejiang University Ningbo Campus, inside the Ningbo Innovation Center, in Ningbo, Zhejiang Province. It was established in May 2025, which makes it a young company, and this was visible in how it works: the office is small, the teams are small, and a student is given a project rather than a task inside someone else's project. Youtune works on software and hardware co-optimisation, AI data products, and edge computing. Edge computing is the part most relevant to what I did. It means running computation on the device where the work happens rather than sending it to a remote server, which matters when a connection is unreliable, when data should not leave the site, or when there is no time to wait for a round trip. The whole of my project follows from that constraint. The company's technical work builds on the System Performance Analytics and Intelligence Laboratory (SPAIL) of Zhejiang University. It was founded by Prof. Kingsum Chow, a faculty member of the same university, and it hosts students for research and internship work carried out together with university faculty. So the office is not a conventional company department. It sits between a company and a university laboratory, and the people working there are a mixture of company staff and students. Supervision of my internship was organised in two levels. Prof. Kingsum Chow, the founder, is responsible for the internship programme in the office as a whole. Prof. Tony Wu of Zhejiang University was my direct technical supervisor: he defined the subject of my project with me on the first day, and I reported to him every week through a meeting and a written progress report in English. I also wrote a longer technical report at the end of each month. I worked in the research and development function of the office. My working days were Monday to Friday, from 09:00 to 18:00 with a one-hour lunch break, which is 40 hours a week, from 6 July to 18 September 2026. That is 55 working days, and this report contains one page for each of them. The equipment I used was a development workstation with an AMD Ryzen AI 9 HX 370 processor and 64 GB of memory. The processor is the relevant part: the whole project was required to run on the CPU, without using the GPU, because the target of the work is hardware with limited resources rather than machines built for this kind of computation. I did not use any laboratory instruments or production equipment, since the work was entirely software. Further details of the company's equipment are not included here, as that information is subject to the company's approval.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 06.07.2026
Yapılan Uygulama:	2. Giriş	
This report describes a system that answers technical questions about electrical engineering documents, running fully offline on an ordinary computer. The problem it addresses is a practical one. Electrical standards, regulations and equipment manuals are long, and they are written to be looked up rather than read through. Someone working with them usually has one narrow question: what ratio applies to this protective device, what clearance is required in front of this equipment, how is this conductor sized. The answer is somewhere in the document, and finding it is the work. A language model can read a passage and explain it, but the models capable of doing this well are normally used through an internet service, which is not always possible. On an industrial site the connection may be poor, the documents may not be allowed to leave the site, and there may be no budget for a service that charges by the question. So the system runs locally, with no internet connection at any point. That decision has a cost: a model running on a CPU takes several seconds to produce an answer, and no setting I tried made it meaningfully faster. Since each answer is expensive and cannot be made cheap, the way to make the system usable is to produce fewer answers. That is what a semantic cache does. It recognises when a new question means the same thing as one already answered, even when the wording is completely different, and returns the stored answer without running the model at all. The system therefore has two parts. Retrieval finds the passages of the documents relevant to a question and gives them to the model, so that the answer comes from the document rather than from what the model happens to remember. The semantic cache sits in front of that and avoids the work entirely when the question has been answered before. The internship divided into two stages, and the report follows them. The first stage built and tested the machinery: the local model servers, the cache and its storage, the checks that stop it returning the wrong stored answer, and the retrieval path. None of that work depends on the subject the system serves. In the second stage, following my supervisor's suggestion, the system was applied to electrical engineering documents, which is where it was measured and where the results in this report come from. The documents used are two United States federal regulations covering wiring design and protection and general electrical requirements, both in the public domain, and a motor equipment manual I wrote myself as a controlled test case. The report is written as one page for each working day, in the order the work happened, including the parts that did not work. Several of the more useful results in it came from measurements that contradicted what I expected, and one problem was open for three weeks before I could explain it. I have written those days as they happened rather than smoothing them, because the way a failure was traced is usually more informative than the fact that it was fixed.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 06.07.2026
Yapılan Uygulama:	Orientation and Definition of the Project Scope	
This was my first day at the office. I met my advisor, Prof. Wu, and we spent the morning defining what I would work on during the internship. The project is about running a large language model on an ordinary computer, fully offline, with no cloud service and no internet connection at run time. Running the model locally means nothing leaves the machine, the system keeps working when the network does not, and there is no cost per question. The price is performance: instead of running on large dedicated hardware, the model runs on whatever processor is available. Every question then takes several seconds of heavy CPU work, and if the same question is asked twice, the machine repeats all of that work for no reason. The idea we agreed on is a semantic cache. A normal cache matches text exactly, so "Where is the university?" and "Where is the university located?" would be stored as two separate entries and the model would be called twice. A semantic cache compares the meaning of the questions instead of the characters, and returns the stored answer when a new question is close enough to one already answered. So the work is not about making the model faster. It is about calling it less often. Prof. Wu set one constraint at the start that shaped everything afterwards: the system must run on the CPU only. The machine has an integrated GPU, but GPU acceleration is out of scope for this project. This makes the problem harder in a useful way. If the GPU were allowed, a slow answer could be fixed by moving work to better hardware, and the caching layer would matter much less. With CPU-only inference, a model call is expensive and stays expensive, so avoiding the call is the only real saving available. Every design decision later in the project came back to this. Three practical questions came out of this discussion, and they turned out to be the work of the whole first week. Which model can run on this machine? How do I run it so that response times are stable enough to compare? And how do I decide that two questions mean the same thing, with no internet connection? The third question is the one that defines the project. Deciding whether two pieces of text are identical is trivial for a computer; deciding whether they mean the same thing is not, because meaning is not in the characters. "What is the current rating of this device?" and "How many amperes can it carry?" share almost no words yet ask the same thing. "Is a 15 A receptacle enough?" and "Is a 30 A receptacle enough?" differ by one character and are completely different questions. A method that counts shared words gets both of these backwards. So the quality of the whole system depends on this one comparison working well, and everything else is built around it. We also agreed on the working rhythm: a short meeting every Friday to go through the week, followed by a written progress report sent by email the same day. I spent the rest of the day meeting the other students working in the office and getting familiar with the working environment.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 07.07.2026
Yapılan Uygulama:	Setting Up the Local Development Enviroment	
Today I prepared the environment I would use for the rest of the internship. The machine is an AMD Ryzen AI 9 HX 370 with 64 GB of RAM. The memory matters here, because the model file has to be held in memory while it runs, and I would later keep two models loaded at the same time. I set up a local build of llama.cpp, on Prof. Wu's recommendation. This is the tool that loads a language model file and runs it directly on the processor, without the large machine learning frameworks that models are normally used with. That is why it suits this project: it is self-contained, so the system does not depend on a long chain of Python packages that could break or try to reach the internet. The build is CPU-only, with no GPU support compiled in, matching the constraint agreed yesterday. Models are loaded in the GGUF format, which packs a whole model into a single file. I also installed Python 3.11 and the packages I expected to need. A large part of the day went to a network problem rather than to the tools. The local network here requires a VPN to reach some of the resources I need, and the connection kept dropping. A program that works sometimes and fails other times, for reasons outside the program, is very hard to debug, so I worked on this until it stayed up reliably. It would have been faster to start writing code and deal with the network later, but I decided against that. If the connection fails at random, every error afterwards has two possible causes: my code or the network. Separating those later, inside a half-finished program, would cost far more than fixing the connection now. This turned out to be right, because a network limitation caused a real problem later in the week, and because I already knew the connection was stable, I recognised the cause immediately instead of suspecting my own code. At the end of the day I checked the environment rather than assuming it worked. I first needed the exact name of the model file, so I listed what was available: Get-ChildItem -Filter *.gguf Select-Object Name With the correct name I loaded the model, ran a question from the command line, and got a sensible answer back. This gave me a known-good starting point: from here on, if something broke, I knew the base setup was not the reason. I also settled the working method I used for the rest of the internship. I edit the scripts in a plain text editor and run them from a PowerShell terminal with python <file>.py, while the model server runs in a second terminal window alongside. The server and my script are separate processes, so I can restart my own code without reloading the model, and watch the server's output in one window while my script prints in the other. Several problems later in the internship were diagnosed by comparing those two windows against each other, including the response-time anomaly that took me two days to find at the end of July.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 08.06.2026
Yapılan Uygulama:	Running the Local Model as a Background Server	
Until today I had been using llama-cli.exe, the basic command line tool that comes with llama.cpp. It works for trying things out, but it loads the whole model into memory every time it runs and unloads it when it finishes. The model file is 2.4 GB, so every single question pays that loading cost from the beginning. This matters more than it first appears, and not only because it is slow. The goal of the project is to measure how much time the cache saves, and that measurement is a comparison between the time taken when the model has to be called and the time taken when a stored answer is reused. If the first number includes several seconds of reading a 2.4 GB file from disk, it is not measuring the model call at all. It is measuring the file system, and any speed-up I reported would be partly an artifact of how the tool starts up. So today I moved to llama-server.exe. This runs the model as a background server that starts once, loads the model once, and stays open. Questions are then sent to it as HTTP requests instead of by starting a new program each time, and the model stays in memory between them. The command is run from the build directory and names the model file, the address to listen on, and the port: <pre>cd "C:\Users\emre\llama.cpp\build\bin\Release" .\llama-server.exe --host 127.0.0.1 --port 8080 -m "<model>.gguf"</pre> The server accepts requests in the same format as the OpenAI API. This was useful in two ways. First, I could call my own local model from Python using an ordinary API client, without writing special code for llama.cpp. The only difference from calling an online service is the address. Second, it keeps the model separate from the rest of the system, so if the model is replaced later the code calling it does not have to change. That separation mattered next week, when I added a second server. There is a cost worth stating alongside the benefit. The server holds the model in memory for as long as it runs, so 2.4 GB is occupied whether or not anyone is asking questions. In exchange, the loading cost is paid once instead of once per question. That is the same reasoning the cache itself is based on: pay a fixed cost once rather than a smaller cost repeatedly. The difference showed up straight away. With the command line tool, every question started from nothing. With the server, only the first request pays a start-up cost and everything after it is consistent. The port number 8080 was just a first choice; I changed it once the system needed more than one server. At the end of the day the model was loaded once and stayed ready between questions, and response times were repeatable enough to measure against. This was the foundation the caching work needed, and I could now start building the layer that sits in front of it.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 09.06.2026									
Yapılan Uygulama:	First Caching Layer and an Offline Similarity Method										
Today I started writing the caching layer. It sits between the user and the model server and works in a simple order. A question arrives; the layer compares it against the questions already stored; if one is close enough it returns the stored answer and stops; if not, it sends the question to the model, stores the question and answer together, and returns the answer. The difficult part is deciding what "close enough" means. My first plan was to use a standard embedding model from Hugging Face. An embedding model turns a sentence into a list of numbers representing its meaning, so two sentences that mean the same thing produce lists pointing in a similar direction, and that similarity can be measured with one calculation. This is the normal way to build a semantic cache. It did not work here. The library tries to download the model files the first time it runs, and the network restrictions and firewall in this location caused constant connection timeouts. The script kept crashing with an OSError before it ever reached my own code. I tried several times, at different times of day, before accepting that this was a property of the environment rather than a temporary failure. It also confirmed something about the project: a system meant to run fully offline should not have a component that needs to download files before it can start. So I wrote my own similarity method in plain Python. It removes the common filler words that carry no meaning on their own and treats what remains as a set of words, comparing how much the two sets overlap to produce a score between 0 and 1. It uses no external library and no internet access. This is clearly weaker than a trained embedding model, and I knew that while writing it. It compares words, not meaning. But the requirement is that the system works offline, and a simple method that always runs is worth more than a better method that fails to start. Replacing the comparison later would mean changing one function, because the rest of the layer does not care how the score is produced. Before connecting the layer to the real model, I tested it on its own. I replaced the model call with a fixed two second delay, so that the only thing under test was the cache logic: does it store the answer, does it recognise the question when it comes back in a different form, and does it return the stored answer instead of running the slow part again. <table><tr><th>Case</th><th>Result</th><th>Latency</th></tr><tr><td>Cache miss</td><td>Slow path executed (2 s stand-in for the model)</td><td>2.0003 s</td></tr><tr><td>Cache hit</td><td>Answer retrieved from memory</td><td>0.0000 s</td></tr></table> The two seconds are artificial, so this says nothing about how fast the model is. That was deliberate: at this stage I wanted to know whether the cache worked, and a real model call would have made that harder to see, because its response time changes from run to run. Testing the two parts separately also meant that when I connected the real model the next day, any problem could only come from the connection between them.			Case	Result	Latency	Cache miss	Slow path executed (2 s stand-in for the model)	2.0003 s	Cache hit	Answer retrieved from memory	0.0000 s
Case	Result	Latency									
Cache miss	Slow path executed (2 s stand-in for the model)	2.0003 s									
Cache hit	Answer retrieved from memory	0.0000 s									
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU									

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih:10.06.2026												
Yapılan Uygulama:	Threshold Calibration and First Timing Measurements													
The cache treats two questions as the same when their similarity score is above a threshold. Choosing that number is a trade-off between two opposite mistakes, and both are real. If it is too high, questions that genuinely mean the same thing score below the limit and the model is called even though the answer is already stored: correct, but slow for no reason. If it is too low, questions that only look similar are matched together and the system confidently returns an answer to a different question. The second mistake is much worse, because a wrong answer delivered instantly looks exactly like a correct one. I started at 60% and ran into the first failure mode: two questions with the same meaning but different wording scored below the limit and were treated as new. I lowered the threshold to 40%, tested again, and the matching behaved as expected. This value is low compared with what a real embedding model would need, but it fits the method I am using, because comparing word sets produces lower scores than comparing meaning. I then connected the layer to the real model server, replacing yesterday's artificial delay with an actual model call, and ran the full pipeline end to end: <table><tr><th>Case</th><th>Question</th><th>Result</th><th>Time</th></tr><tr><td>Cache miss</td><td>"Zhejiang University is a prestigious university located in"</td><td>Model called</td><td>7.5640 s</td></tr><tr><td>Cache hit</td><td>"Where is Zhejiang University located?"</td><td>Stored answer reused, similarity 0.5000</td><td>0.0000 s</td></tr></table> The second question is written differently from the first, in a different grammatical form, but the cache recognised it and returned the stored answer without calling the model at all. The 0.0000 second reading does not mean the work was instant; it means the lookup finished faster than the timer could measure, which is itself the point. Against roughly 7.5 seconds for a model call, the cost of checking the cache is too small to be worth measuring. It is worth noting that this number comes from avoiding the model, not from making anything faster. This is the behaviour the whole project is built on, and seeing it work end to end was the main result of my first week. At the end of the day I wrote my weekly progress report and sent it to Prof. Wu, together with the console output above as evidence. Next step: the cache currently exists only in memory, so everything is lost when the script stops. Next week I will make it permanent by storing it in a local SQLite database.			Case	Question	Result	Time	Cache miss	"Zhejiang University is a prestigious university located in"	Model called	7.5640 s	Cache hit	"Where is Zhejiang University located?"	Stored answer reused, similarity 0.5000	0.0000 s
Case	Question	Result	Time											
Cache miss	"Zhejiang University is a prestigious university located in"	Model called	7.5640 s											
Cache hit	"Where is Zhejiang University located?"	Stored answer reused, similarity 0.5000	0.0000 s											
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU												

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 13.06.2026
Yapılan Uygulama:	Attempting to Integrate the GPTCache Library	
At the end of last week the cache existed only in memory, so everything was lost when the script stopped. My plan, written in last week's progress report, was to fix this by integrating GPTCache, an existing open source library built for exactly this purpose: caching the answers of language models and matching questions by meaning rather than by exact text. Using a maintained library instead of writing my own would have saved time and given the project a more standard foundation. I spent today trying to make it work, and it did not. I hit three separate problems. The first was version compatibility. I was working with version 0.1.44, and several parts of it did not behave as the documentation described on my Windows setup. The second was more serious for this project specifically: the library expects to reach Hugging Face Hub over the internet to fetch the components it uses for similarity. This is the same limitation I ran into last Thursday with the embedding model, and it is not something I can configure away, because the online lookup is built into how the library initialises itself. The third problem was that even when I got past the first two, calls into the library failed inside its own code while unpacking its internal return values, which left me debugging a library I did not write rather than the system I was supposed to be building. At that point I stopped and considered what I actually needed. The requirement is a cache that stores questions with their answers, compares a new question against the stored ones, and survives between runs. That is a small amount of well understood work. GPTCache is a much larger and more general library, and most of what makes it large is support for online services and for storage backends I will never use. I was paying the full cost of that generality while being blocked by the one assumption in it that I cannot satisfy, which is internet access. I did weigh what I would be giving up, because writing your own version of an existing library is usually the wrong instinct. A maintained library has been tested by many more people than I could test it with, it handles cases I have not thought of yet, and using one means I can point to a known component instead of defending my own code. Those are real advantages and I would rather have had them. What decided it was that none of those advantages survive if the library cannot start. A tested component that fails at initialisation on this machine gives me nothing. And the specific thing blocking it is not a bug that might be fixed in a later version, or a setting I have not found yet: it is the library reaching out to the internet by design, in an environment that is defined by not having internet. That conflict does not go away with more effort. So I decided to write my own cache instead. This was not a preference for writing code from scratch. It was that the library's central assumption conflicts with the central requirement of this project, and no amount of configuration changes that. The work I lose is a day; the work I gain back is not having to fight a component I do not control every time something breaks. Next step: build the replacement tomorrow, with permanent storage as the first requirement.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 14.07.2026												
Yapılan Uygulama:	Writing a Custom Offline Cache with Permanent Storage													
Today I wrote the replacement for GPTCache, a class called LocalSemanticCache, and this is the component the rest of the project was built on. It stores everything in a local SQLite database file, semantic_cache.db. SQLite is a good fit here: it is a single file, it needs no server running in the background, and it comes with Python, so it adds no dependency and no network access. Each entry holds the original question, the answer, and the vector that represents the question's meaning: <pre>cache(id, question, answer, embedding BLOB, last_accessed)</pre> The vector is stored as a BLOB, meaning raw bytes, because SQLite has no column type for a list of numbers, so it has to be converted on the way in and back on the way out. I did not think much about this at the time. It mattered a great deal a month later, when I measured where the time in a search actually goes. Storing the vector in a database forced a change I had not planned. Last week I compared questions as sets of words, but a set of words has no fixed size and cannot be stored or compared efficiently as a database column. So I replaced it with a function that turns any question into a fixed length list of 384 numbers, produced directly from the characters and words of the text. It is fully deterministic and uses no trained model, so it runs offline and always produces the same vector for the same question. With every question represented as a vector of the same length, two questions can be compared with cosine similarity, which measures the angle between two vectors and returns a score between 0 and 1. I set the matching threshold to 0.55. I also added an eviction policy. A cache that only grows will eventually fill the disk, so when it reaches its limit it removes the entry that has not been used for the longest time. This meant tracking when each entry was last read, in a last_accessed column. Since I already had a database file with entries in it, I wrote the code so that it checks whether that column exists and adds it if it does not, instead of failing on an older database. This is a small thing, but it meant I never had to delete my test data to move forward. In the afternoon I wrote a separate test script to check whether the new similarity function was actually sound, rather than assuming it was. I tested it on pairs I already knew the right answer for: words made of the same letters in a different order, two completely unrelated sentences, and two questions asking about the same subject in different words. <table><tr><th>Question 1</th><th>Question 2</th><th>Similarity</th></tr><tr><td>"listen"</td><td>"silent"</td><td>0.00%</td></tr><tr><td>"Ohm's law formula"</td><td>"Apple pie recipe"</td><td>0.00%</td></tr><tr><td>"Explain Kirchhoff's current law"</td><td>"Tell me about Kirchhoff's laws"</td><td>44.72%</td></tr></table> The first two results are correct. The third is the problem. Those two questions are asking about the same thing and should match, but 44.72% is below my 0.55 threshold, so the cache would call the model again for a question it had already answered.			Question 1	Question 2	Similarity	"listen"	"silent"	0.00%	"Ohm's law formula"	"Apple pie recipe"	0.00%	"Explain Kirchhoff's current law"	"Tell me about Kirchhoff's laws"	44.72%
Question 1	Question 2	Similarity												
"listen"	"silent"	0.00%												
"Ohm's law formula"	"Apple pie recipe"	0.00%												
"Explain Kirchhoff's current law"	"Tell me about Kirchhoff's laws"	44.72%												
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU												

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 15.07.2026
Yapılan Uygulama:	Adding a Real Embedding Model as a Second Local Server	
Yesterday's test showed that comparing characters and words is not enough. The function has no idea what a sentence means; it only sees the letters. Two questions about the same subject scored too low, and worse, two questions that are structurally alike but ask about different things scored too high. So today I replaced it with a real embedding model. The problem was how to run one without internet access, since that is exactly what blocked me last week. The solution came from something already working in the project: llama.cpp does not only run chat models, it can also run embedding models, and it loads them from a local file. So I downloaded all-MiniLM-L6-v2 in the GGUF format directly, placed the file on the machine, and started it as a second llama-server instance: <pre>.\llama-server.exe -m "all-MiniLM-L6-v2-Q8_0.gguf" --embedding --pooling mean --port 8001</pre> Two of those flags are what make this work. --embedding tells the server to expose the endpoint that returns vectors instead of generated text, so it stops behaving like a chat model. --pooling mean decides how the model's per-word outputs are combined into one vector for the whole sentence, by averaging them. Without it there is no single vector to compare, because the model produces one output per token rather than one per sentence. This gave the project the two server structure it kept for the rest of the internship. One server generates answers, the other turns questions into vectors, and each does one job. This is also where the port numbers changed. Last week I had started the first server on port 8080, which was just the first number I picked. With two servers running at once, I renumbered them to 8000 for generation and 8001 for embeddings, so that the two would not be confused with each other in code or in the logs. The important part of this choice is what it avoided. The normal way to use this model in Python is through libraries like sentence-transformers and PyTorch, which are large, slow to install, and try to download files at first use. By serving the model through llama.cpp instead, I added a real machine learning model to the system without adding a single Python package. The vectors are requested over HTTP from the local machine, in the same API format the other server already uses, so the code that asks for them is a few lines long. The model produces vectors of 384 numbers, which is the same length as the ones my character based function produced. That coincidence is convenient, because nothing in the database schema or the comparison code had to change, but it is also a trap that cost me the whole of the next day. I hit one error while starting the server: gguf_init_from_file: failed to open GGUF file. The cause was simply that I had given a model path that did not point at the file. I moved the model file into the same directory as the server executable and referred to it by name, and the server started correctly. At the end of the day the embedding server was answering requests, and the pipeline was using real 384 dimensional semantic vectors instead of the character based ones.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 16.07.2026
Yapılan Uygulama:	Diagnosing Persistent Wrong Cache Hits	
Today started with a result that did not make sense. I had replaced the similarity method with a real embedding model, but the system was still returning wrong answers from the cache in the same way as before. The clearest example is one I had been using as a test case: after asking who founded the Turkish Republic and storing the answer, asking who founded the Bulgarian Republic returned the stored Turkish answer. Two questions about two different countries, and the cache treated them as the same question. If the behaviour does not change after replacing the component you believe is responsible, then either the component was not the cause or it is not actually being used. So instead of changing more code, I wrote a small diagnostic script to measure what was really happening. It found two causes, and neither was the one I expected. The first was the database. The old entries were still in semantic_cache.db, created with the old character based method, and new questions were being compared against those old vectors. Both methods produce exactly 384 numbers, so nothing crashed and no warning appeared anywhere: the system was comparing a real semantic vector against a character hash and reporting the result as a similarity score. The fix was to delete the database file. The lesson is that the coincidence which made yesterday's integration painless is the same coincidence that hid this bug; different vector lengths would have crashed the program immediately. The second was mathematical. Cosine similarity is only a similarity between 0 and 1 if both vectors have length 1. I was calculating it as a dot product without checking that, and the vectors coming back from the embedding server were not exactly unit length, so the result could rise above 1.0 and pass any threshold I set. The fix was to divide each vector by its own length before storing it, which is called L2 normalisation: <pre>python norm = np.linalg.norm(vector) return vector / norm if norm > 0 else vector</pre> The check for a zero norm is there because dividing by zero would produce invalid numbers that spread silently through every later comparison. A zero length vector should not happen, but the safe branch is worth one line. I verified afterwards that the stored vectors had a length of 1.0000.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 16.07.2026
Yapılan Uygulama:	Diagnosing Persistent Wrong Cache Hits (continued)	
With both fixed, I measured the actual similarity of the two founder questions: 74.49%. This is a genuinely high score, and the model is not wrong to give it. The two sentences share almost all of their words and their entire structure. They differ by one word, and that word happens to carry the whole meaning. So the last step was to raise the matching threshold from 0.55 to 0.80. At 74.49%, the two questions now fall below the line, and the system correctly reports a cache miss and asks the model. It is worth noting how narrow that margin is: 74.49% against a limit of 80% leaves about five and a half points. A slightly different pair of countries, or a slightly different sentence, could easily land on the wrong side. Raising the threshold solved today's case rather than the class of problem behind it, and I came back to this the next day. I also began writing a development report today, setting out where the project stands after these first two weeks. I did not finish it, and carried it over.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 17.07.2026
Yapılan Uygulama:	Building the First Retrieval Pipeline (RAG)	
With the cache working correctly, I moved on to the part Prof. Wu had asked about: retrieval augmented generation, or RAG. Instead of relying on whatever the model learned during training, the system stores documents, finds the passages relevant to a question, and gives them to the model as context. This is made of four pieces. I started the first the day before and finished the rest today, reusing what already existed wherever I could. First, storage. I added a second table, documents, to the same database, holding each piece of text, its embedding vector, and the name of the file it came from. That last column matters more than it looks: without it there is no way to tell later which document an answer came from. Second, splitting. A whole document cannot be given to the model at once, and a single vector for a long text represents nothing useful. So I wrote a function that cuts a document into overlapping pieces of a fixed number of words. The overlap is deliberate: if a piece ends mid-rule, the next begins slightly earlier so the rule is complete in at least one of them. Third, retrieval. A question is turned into a vector by the same embedding server the cache uses and compared against every stored piece, and the closest few are selected. This is the calculation the cache already does, applied to a different table, so there was no new logic to get wrong. Fourth, the prompt. The retrieved pieces go into a prompt telling the model to answer using only the information given, which is sent to the generation server on port 8000. I also added a step that runs before the cache lookup. Users write the same question in many ways, so the system first asks the model to rewrite it into a standard form, and the cache is searched and stored using that form while the answer is still generated from what the user actually wrote. This creates a risk of its own, so I added a guard next to it. Standardising two questions makes them more alike by design, which pushes questions that should not match closer together too. Until now the only protection was the similarity threshold, and yesterday showed how narrow that margin is: 74.49% against a limit of 80%. So before accepting a match, the system now also extracts the named entities from both questions and rejects the match if they disagree, however high the score. The two fail in different ways, so requiring both is safer than trusting either alone. At the end of the day the full path worked: a question goes in, relevant document text is found, and the answer is built from it. I had my weekly meeting with Prof. Wu and sent my progress report. Next step: test whether the rewriting step actually improves matching, since it adds a model call to every question.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 20.07.2026
Yapılan Uygulama:	Testing Question Standardisation and Fixing the Entity Check	
On Friday I added a step that rewrites each question into a standard form before the cache is searched. Today I tested whether it actually works, because it adds a model call to every question and that cost is only worth paying if the matching genuinely improves. The test case was simple. The cache already held an answer for "Who is the founder of the Turkish Republic?". I asked "Tell me the founder of Turkey", which is the same question in different words, and expected a cache hit. It missed. Looking at what the standardisation step produced explained why. It tidied up the sentence structure, which is what I asked it to do, but it left the country name exactly as written. "Turkish Republic" in one question and "Turkey" in the other stayed as two different names, so the two standardised questions were still different where it mattered most. There is a second check in the cache that made this worse. Before accepting a match, the system compares the named entities in the two questions and rejects the match if they disagree. This guard exists because of the wrong-answer problem from last week: two questions can be almost identical in wording while asking about two different subjects, and the entity check is what stops the cache from confusing them. Here it saw "Turkish Republic" in one question and "Turkey" in the other, decided they were different subjects, and rejected the match. The guard was working exactly as designed, and the result was still wrong. My first attempt at a fix was to extend the standardisation prompt, asking the model to also normalise names of places and organisations. I decided against keeping this. It makes the correctness of the cache depend on the model following an instruction, and this model does not always follow instructions the same way twice. A cache that is right most of the time is a bad cache, because the whole point of it is to be trusted without checking. So I reverted the prompt to what it was and solved the problem in the entity check instead. I added a small lookup table that maps an adjective form of a name to the name itself, and applied it inside the function that extracts entities from a question. Now both questions produce the same entity before they are compared. This runs the same way every time, costs nothing, and does not involve the model at all. After the fix I reran both cases. "Tell me the founder of Turkey" now returns a cache hit at 86.10% similarity, matching the stored entry. The important part is the second test: the earlier Turkish and Bulgarian case still correctly misses. Checking both directions was deliberate. Every change to a cache moves it along one axis, from stricter to looser or the other way, and a change that fixes a missed match will also, by the same mechanism, make some wrong matches more likely. Testing only the case I was trying to fix would have told me the change worked without telling me what it cost. Here the answer was that it cost nothing, because the alias table only makes two names for the same country identical and does not touch two different countries. But I only know that because I checked. This also answers the question I started the day with. The standardisation step is worth keeping: it does improve matching, and the case where it failed was not its fault but a gap in what the entity check considered the same name.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 21.07.2026
Yapılan Uygulama:	Narrowing the System to a Single Subject Area	
Up to now the system would accept any question at all. Today, following Prof. Wu's suggestion, I narrowed it to a single subject area. The area I picked at this stage was a development and testing choice; the system was moved onto its final subject later in the internship. The reason for narrowing matters more than which area was chosen. A semantic cache is only useful when questions repeat, and questions only repeat when they come from a limited field. If the system accepts anything, users ask about anything, almost every question is new, and the cache sits nearly empty while the model is called every time. Narrow the field and the opposite happens: the same small set of subjects is asked about repeatedly in different words, which is exactly the situation the cache is built for. The same argument applies to retrieval, because documents can only be collected for a field that has been defined. Narrowing also makes the system safer to evaluate. With an open ended system there is no way to say whether an answer is right, because there is no defined set of documents it should have come from, and no way to distinguish a wrong answer from a question the system was never meant to handle. Once the field is fixed, both of those become clear: a question is either inside the field or outside it, and an answer inside the field is either supported by the documents or it is not. A wrong answer becomes a measurable failure rather than an opinion. This became important later when I built the accuracy measurements. The change touched three parts of the system. First the system prompt, which now tells the model what kind of assistant it is and what subject it answers questions about, instead of leaving it general. Second the documents stored for retrieval, which were replaced with material from the chosen field. Third the test questions I use while developing, which had been a mixture of unrelated examples and now all come from one field, so that a test result actually tells me something about the intended use. The third of these turned out to matter most day to day. Until now my test questions were whatever came to mind, which meant a passing test proved only that the system handled that particular question. Questions from a single field share vocabulary and structure, so they exercise the parts of the system that a mixed set never touches, in particular the cases where two questions are genuinely close to each other. Most of the bugs I found over the following weeks were found this way. None of this changed the architecture. The cache, the two servers, the entity check and the retrieval path all stayed exactly as they were. What changed is the content flowing through them: which documents are loaded, what the prompt says the assistant is, and which questions I test with. That separation between the machinery and the subject it serves is the reason the same system could be moved to a different field later without being rewritten, which is what happened in August. Next step: with a defined field, it becomes possible to guess what users will ask before they ask it, and to prepare those answers in advance. This is the next task Prof. Wu set.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 22.07.2026
Yapılan Uygulama:	Building a Proactive Cache Warming System	
Until now the cache only learned from questions that were actually asked. The first person to ask any given question always waits the full model time, and only later users benefit. Prof. Wu asked me to change this: since the machine is idle at night, it can spend that time generating and storing answers in advance, so that even the first user gets an instant answer. I built this in four steps. First I defined question templates with a slot in them, so that one template describes a family of questions rather than a single question. Then I listed the entities that could fill that slot. Ten templates across sixteen entities gives 160 combinations, and each of those is a plausible question a user might ask. Second I added a priority weighting, because not all combinations are equally likely to be asked. There is no point spending the night generating answers nobody will want, so the more likely combinations are generated first. This also means that if the process is interrupted, or if I stop it early because I need the machine, the most valuable work is already done rather than half of an arbitrary list. Third I wrote the warming routine itself. The important decision here was to reuse the existing cache search rather than write a new one. Before generating an answer, the routine asks the cache whether this question is already covered, using the same function a live user's question goes through. This has two benefits: it does not waste model time regenerating answers that exist, and it means the warming process and the live path can never disagree about what counts as a match, because they run the same code. Fourth I added a small interactive mode to the program, so I could type questions directly and watch the result instead of editing the script every time I wanted to test something. This was for my own convenience, but it saved a great deal of time for the rest of the internship, and it is also how I noticed several problems that a scripted test would not have surfaced. A full warming run takes about 25 minutes for the 160 combinations and runs unattended. That figure is itself informative: 160 model calls in 25 minutes is roughly nine seconds each, which matches the response time I measured in week one and confirms that the model call really is the entire cost of the system. There is a limit to this approach that is worth stating, because it is easy to oversell. Warming only helps for questions somebody actually thought to write a template for. A user who asks something outside the prepared set still waits the full model time, exactly as before. What warming does is move the most predictable part of the workload out of the user's waiting time and into the machine's idle time. It does not make the system faster in general; it makes the common case free and leaves the uncommon case unchanged. Next step: run the warming process on the real server and check that what it stored is actually correct.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 23.07.2026
Yapılan Uygulama:	Finding and Fixing a Topic Collision in the Cache	
Today I ran the warming process against the real servers and checked what it had stored, rather than assuming it had worked because it finished without an error. Generating 160 answers automatically means that if something is wrong, it is wrong 160 times, so this check mattered. It had not worked. Several questions had not been generated at all, because the cache reported them as already covered. That is a suspicious result on a fresh set: if these questions were genuinely new, nothing should have matched them. Looking at what they had matched against showed a real bug. The entity check I fixed on Monday compares the named entities in two questions. That is what it does, and that is all it does. It has no idea what the question is asking <i>about</i>. So two questions naming the same entity but asking about two completely different aspects of it passed the entity check without difficulty, and their similarity scores landed at 70 to 76%, comfortably below my 0.80 threshold but close enough that the standardisation step could push some of them over. The result was that a question got an answer belonging to a different question about the same subject. This is the same class of error as the Turkish and Bulgarian case from last week, and I had thought I had solved that class. I had not. I had solved it in one dimension only. Two questions can differ by their subject, which the entity check catches, or by what they ask about that subject, which nothing was checking. The fix was a second, separate gate rather than a change to the first one. I built a small vocabulary of keywords for each topic in the field, and wrote a function that works out which topic each question belongs to and rejects the match if the two disagree. I deliberately kept it separate from the entity check instead of extending that function. Two reasons. First, each gate then does one job, and when a match is rejected I can see which gate rejected it, which matters when the system is behaving in a way I did not expect. Second, they can be reasoned about independently: the entity check asks "is this about the same thing?" and the topic check asks "is it asking the same thing about it?". Merging them into one function that answers both would make either question harder to change later without disturbing the other.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 23.07.2026										
Yapılan Uygulama:	Finding and Fixing a Topic Collision in the Cache (continued)											
I reran the warming process to confirm the fix on the real server: <table><tr><td>Result</td><td>Count</td></tr><tr><td>Combinations attempted</td><td>40</td></tr><tr><td>Already cached</td><td>0</td></tr><tr><td>Newly cached</td><td>40</td></tr><tr><td>Records stored</td><td>40</td></tr></table> The eight questions that had previously been given the wrong stored answer now each generate and keep their own correct answer. The wider lesson is about how this bug was found. It did not appear as an error or a crash. It appeared as a count that was slightly lower than it should have been, in a process I could easily have marked as successful because it finished without complaining. Checking what an automated process actually produced, rather than that it completed, is what turned an invisible wrong answer into a fixed one.			Result	Count	Combinations attempted	40	Already cached	0	Newly cached	40	Records stored	40
Result	Count											
Combinations attempted	40											
Already cached	0											
Newly cached	40											
Records stored	40											
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU										

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 24.07.2026
Yapılan Uygulama:	Speeding Up Cache Hits and Investigating an Idle Delay	
A cache hit is supposed to be almost free. Measuring the system today showed that hits were taking nearly as long as misses, which defeats the purpose of having a cache at all. The cause was the standardisation step I added last week. Every incoming question was being rewritten by the model <i>before</i> the cache was searched. That rewrite is itself a model call, so even a question that was already in the cache had to wait for the model before it could be found. I had put an expensive step in front of a cheap one. The fix was to change the order. The system now searches the cache with the raw question exactly as the user typed it. Most repeated questions are typed similarly enough to be found this way, and when that succeeds the answer comes back with no model call at all. Only when the raw question finds nothing does the system pay for the rewrite and search again. The standardisation still catches the harder rewordings, but it is no longer charged to every question, only to the ones that need it. This structure, a cheap check first and an expensive one only on failure, is now the shape of the whole lookup path. The rest of the day went to a problem I did not solve. If the system sits idle for more than about ten seconds, the next question takes roughly 2 seconds instead of the usual 0.01 seconds, even when it is a cache hit that should not involve the model at all. After that first slow question, everything is fast again until the next idle period. I tried to narrow it down by eliminating explanations. I checked whether the model server was reusing its internal processing slots in a way that required setup after a pause, and ruled that out. I checked whether the processor was parking its cores when idle and needed time to wake them, and ruled that out too. Neither explanation survived testing. What I can say from the measurements is that the delay depends on elapsed idle time and not on what the question is, and that it affects requests that never reach the model. That points away from the model and towards the connection between my code and the servers. I will continue on the operating system and network side next week. I had my weekly meeting with Prof. Wu and sent the progress report.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 27.07.2026						
Yapılan Uygulama:	Finding the Cause of the Idle Delay							
I came back to the problem I could not solve on Friday. After the system sits idle for about ten seconds, the next question takes around 2 seconds instead of 0.01 seconds, even when it is a cache hit that never touches the model. What I already knew narrowed the search. The delay does not depend on the question, so it is not about the content. It affects requests that never reach the model, so it is not the model. It depends on how long the system has been idle, so something expires during that idle time. I had already ruled out the model server reusing its internal slots, and the processor parking its cores. That left the connection itself. The cause was one word in my own code. I was sending requests to localhost. That is a name, so it has to be resolved into an address before a connection can be made, and on this machine it resolves to the IPv6 address first. My servers only listen on IPv4. So every new connection tried IPv6, waited for it to fail, then fell back to IPv4. When requests come quickly the connection is already open and reused, so the cost is invisible. After an idle period the connection has been closed, and the next request pays the full failed attempt. The fix was to replace localhost with 127.0.0.1 in both clients, the one for the model server and the one for the embedding server. That is an address, not a name, so there is nothing to resolve and nothing to fail. <table><tr><td></td><td>Before</td><td>After</td></tr><tr><td>Cache hit after idle period</td><td>~2.0–2.5 s</td><td>~0.02–0.03 s</td></tr></table> The result is now stable no matter how long the system has been idle. That matters more than the ratio. A system that is only fast while it is being used constantly is not really fast. I want to record what this cost me. I spent most of Friday looking inside the model server and the processor, because that is where I assumed a delay like this would come from. The real cause was in two or three characters I typed weeks earlier without thinking about them. The measurements are what saved me. Because I could show the delay hit requests that never reached the model, I could rule the model out no matter how plausible it looked. Without that, I would probably still be tuning server settings. It is also worth noting that this problem was invisible while I was working. Testing means asking questions one after another, and the connection stays open, so the delay never appears. It only shows up for a real user, who asks a question, reads the answer, and asks the next one a minute later.				Before	After	Cache hit after idle period	~2.0–2.5 s	~0.02–0.03 s
	Before	After						
Cache hit after idle period	~2.0–2.5 s	~0.02–0.03 s						
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU						

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 28.07.2026
Yapılan Uygulama:	Rewriting the Similarity Search as a Matrix Operation	
In the morning I tried tuning the model server, mainly the thread count and batch size, to see if the remaining response time could come down. It made no clear difference at this scale. With yesterday's connection problem gone, what is left of a slow question is the model computing the answer, and no setting I tried changed that. I recorded the result and moved on, since Prof. Wu had asked me to work on vectorized similarity next. Until today, comparing a question against the cache worked one row at a time. Fetch a stored vector, calculate its similarity, keep it if it is the best so far, move to the next. This is the obvious way to write it and it is correct, but it repeats the same small calculation hundreds of times in a Python loop, and Python is slow at exactly that. The alternative is to stop treating the stored vectors as a list and treat them as a matrix. If every stored vector is a row, then multiplying that matrix by the question vector gives all the scores in one operation, calculated inside NumPy instead of in Python. The loop disappears. The arithmetic is exactly the same; what changes is that it runs in compiled code once rather than in interpreted code many times. I applied this to three places that were all doing the same thing in slightly different ways: the cache search, the function that reports the best matching score, and the document retrieval used by RAG. The important part is that this must not change any result. A rewrite that is faster but decides differently is not an optimisation, it is a new system. So I ran the new version against the old one on the same data and confirmed the scores and the hit and miss decisions came out identical. Mathematically equivalent code does not always behave equivalently in practice, so I checked instead of assuming. I also made a decision here that I came back to later. Cosine similarity is only a true cosine when both vectors have length exactly 1. My vectors are normalised when they are stored, so they are already close enough that adding another normalisation step inside the matrix calculation would change the numbers by too little to affect any decision. I left it out, because the behaviour was stable and the extra work was not buying anything. I revisited this in August. There is one more reason this change matters beyond speed. The loop version got slower as the cache grew, in direct proportion to the number of stored entries. That puts a ceiling on how large the cache can usefully become, because at some point searching it costs more than the model call it is meant to avoid. The matrix version pushes that ceiling much further out. Next step: measure the actual speed difference, and grow the cache enough that the measurement means something.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 29.07.2026															
Yapılan Uygulama:	Expanding the Prepared Question Set to 600 Combinations																
The warming system built last week covered 10 question templates across 16 entities, which is 160 combinations. Prof. Wu asked me to widen it. A larger set covers more of what a user might ask, and it also gives me a bigger cache to test the new matrix search against. The 160 entry version was too small to show the difference properly. I added 20 new templates and 4 new entities, taking the set to 30 templates across 20 entities, which is 600 combinations. Adding templates is not just typing more questions. Each new template brings a new topic, and the topic check decides whether two questions are about the same thing by looking for topic keywords. More topics means more chances that two keyword lists overlap, and an overlap means the check can no longer tell those two topics apart. The risk grows faster than the number of topics does, because what matters is the number of pairs, not the number of topics. That is what happened. Two of the new topics both matched on a broad word that neither of them owns, so questions from one were being classified as the other. I found it by checking the topic assignment of the new templates, rather than waiting for a wrong answer to show up. The fix was to drop the over-general keyword and give one topic a more specific word that only it uses. I then ran the warming process against the real servers: <table><tr><td></td><td>Result</td><td>Count</td></tr><tr><td>Combinations attempted</td><td></td><td>600</td></tr><tr><td>Already cached</td><td></td><td>17</td></tr><tr><td>Newly cached</td><td></td><td>583</td></tr><tr><td>Records stored</td><td></td><td>583</td></tr></table> The 17 already-cached entries needed explaining before I could call this a success, because "already cached" is also what a wrong duplicate match looks like. Last week the same number told me about a real bug, so I could not treat it as noise. I checked three things. Whether any question appears twice in the set. Whether any pair of topic keywords could collide across all 20 entities. And whether any topic ended up with no keywords at all, which would make its check pass automatically. All three came back clean, so the 17 are most likely left over from my own earlier interactive testing sessions, which used the same database. At 600 entries the cache is now large enough that the matrix search has something real to be measured against, which was the second reason for widening it.				Result	Count	Combinations attempted		600	Already cached		17	Newly cached		583	Records stored		583
	Result	Count															
Combinations attempted		600															
Already cached		17															
Newly cached		583															
Records stored		583															
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU															

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 30.07.2026
Yapılan Uygulama:	Translating the Code to English and Preparing Report Evidence	
Today was preparation for the monthly report Prof. Wu asked for, plus one piece of housekeeping. The housekeeping was language. I had written the comments, docstrings and console messages in Turkish, because I was the only person reading them. That stopped being true once the project became something I report on. My advisor reads English, the reports are written in English, and console output is the main evidence I attach to them. A screenshot with Turkish labels is not usable in an English report, and a comment nobody else can read is not doing its job. So I went through the code and translated all of it. Nothing about the behaviour changed, which made this easy to keep putting off. The reason I did it now is that the cost only grows: the longer I wait, the more there is to translate, and the more likely I am to need a screenshot in a hurry and find it unusable. The rest of the day went to the report. Two things had to be done before any writing. The first was the architecture diagram. Until now the design existed as separate pieces I understood one at a time. Drawing it forced me to lay out the full path a question takes: in through the cache lookup, out immediately on a hit, or through retrieval and the model on a miss, then stored and returned. This was more useful than I expected. It made the shape of the system visible in one picture, and it showed which parts were really connected and which I had only assumed were. The second was collecting evidence. Through the month I had been reading measurements off the console and moving on. For the report I needed the output kept and organised, so I re-ran the things I wanted to show and saved the real console output instead of describing results from memory. Now every number in the report can be traced to output I actually produced. It also caught something: some of what I remembered as measured had only been noticed in passing, which is not the same thing. Drawing it also settled a question of scope. The diagram has a boundary, and deciding what sits inside it meant deciding what this project is. The model servers are inside, because I start and configure them. The model itself is outside, because I did not train it and cannot change how it reasons. That line matters for the report: it separates what I can claim as my work from what I am building on top of. One thing the diagram made clear was how much of the system exists to prevent a wrong answer rather than to produce a right one. The path that generates an answer is short: look up, miss, retrieve, ask the model, store, return. Almost everything else on the diagram is a check standing between a match and its acceptance. That balance is not something I planned; it is what four weeks of finding wrong matches produced.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 31.07.2026
Yapılan Uygulama:	Writing and Submitting the Monthly Progress Report	
Today I wrote the monthly progress report and submitted it to Prof. Wu. He asked for a fixed six section structure: objectives, background, related work, the solution, the experimental study, and the conclusion. Writing to a required structure instead of my own order was harder than I expected, and useful for the same reason. The related work section in particular made me look at what already exists in this area, rather than only describing what I built. That is where the GPTCache decision from week two came back. I had to explain not just that I wrote my own cache but why the existing library did not fit, in terms someone outside the project would accept. "It did not work for me" is not an answer there. The answer is that its central assumption, downloading components over the internet, conflicts with the central requirement of this project, which is running fully offline. I went through the draft twice for two specific things. The first was person and tone. My first draft used "we", which is normal in academic writing but not accurate here, since this is my own work under supervision. I changed it to first person throughout. I also shortened the long sentences. Neither Prof. Wu nor I have English as a first language, and a sentence that has to be read twice is a worse sentence for this purpose. The second was evidence. I included yesterday's architecture diagram, and an appendix with real console output from the actual runs, so a reader can check the results rather than take my word for them. Filling the experimental study section was the part that showed me what I actually have. Writing "the matrix search is faster" and then looking for the measurement to support it is a good way to find out that the measurement does not exist. Some of what I wanted to claim turned out to rest on a result I had seen once and not recorded, so I either re-ran it and saved the output or left the claim out. That is a slower way to write a report, but the alternative is a report I cannot defend. Looking back over the month, the progress is easier to see written down than it was day to day. The month started with a cache that did not survive being closed and compared questions by counting shared words. It ends with one that stores real semantic vectors in a database, protects itself against two separate kinds of wrong match, searches by matrix operation instead of by loop, fills itself during idle time, and answers a repeated question in about a hundredth of a second. I had my weekly meeting with Prof. Wu and submitted the report. He asked me to focus next on the vectorized similarity work, specifically on making the matching both faster and more consistent.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 03.08.2026
Yapılan Uygulama:	Testing Thread Settings to Reduce Cache Warming Time	
The cache warming process now runs unattended, but it is not free. Every combination it prepares is one full model call, and the model call is the entire cost of the system, as I measured in week one and confirmed in week three. The arithmetic is worth setting out, because it is what made this worth a day's work. In week three the warming run covered 160 combinations in about 25 minutes, which is roughly 9 seconds for each. At the current set of 600 combinations, the same rate gives about 90 minutes for a full warming run. That is not a problem when it runs overnight, but it becomes one as soon as I want to change the question set and see the result, because every change costs an hour and a half before I can look at anything. So today I went back to the model server settings, this time aiming at throughput during warming rather than at the response time of a single question. That distinction matters. A user asking one question cares how long that question takes. A warming run cares how many questions get through in an hour. Those are not always improved by the same settings, which is why it was worth trying again after the general tuning on 28 July had produced nothing. The main setting I worked on was the thread count. The model runs on the CPU, and llama.cpp splits the arithmetic of generating each token across several threads. Setting this too low leaves cores unused. Setting it too high does not keep helping, because the threads start competing for the same memory bandwidth and for cores that other parts of the system also need, including my own embedding server, which is a second process on the same machine. I tried a range of thread settings and measured the effect on processing speed. The result was that no setting produced a clear improvement at this scale. I had already found something similar on 28 July when tuning for single-question latency, and the same thing held here for throughput. This is a negative result, but a useful one. It means the model call time is not something I can configure away on this hardware. Every further improvement has to come from calling the model less often, not from making each call cheaper. That is the direction the project was already going, and this closes off the alternative rather than leaving it as an untested possibility. It also puts a floor under the numbers I report later. If model time could be halved by a setting, then any speed-up I measure from caching would depend on which setting I happened to be using. Knowing it cannot means the comparison between a cache hit and a cache miss is stable. Next step: widen the prepared question set, which is the other lever on how much of a user's questions the cache can already answer.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 04.08.2026
Yapılan Uygulama:	Expanding the Prepared Question Set to 1,000 Combinations	
Today I expanded the prepared question set from 30 templates across 20 entities to 40 templates across 25 entities, taking it from 600 combinations to 1,000. There were two reasons for going wider. The first is coverage. The more of the plausible question space is prepared in advance, the more likely it is that a real user's first question is already answered and comes back instantly. Everything the warming system does is aimed at that first user, since every user after them would have hit the cache anyway. The second, stated in my weekly report, is testing. A larger prepared set is also a larger set of questions I can search against, and some of the behaviour I want to check only shows up at a size the smaller set never reached. Expanding the set is not only a matter of writing more questions. Every new template brings a new topic with it, and the topic check compares questions by looking for topic keywords. The number of topic pairs that could overlap grows faster than the number of topics: going from 30 templates to 40 adds ten topics, but it adds far more than ten new pairs that have to stay distinguishable from each other. I had already hit exactly this problem on 29 July, when two of the newly added topics matched on a word that was too general and had to be separated. The cost side also grows. At the rate measured in week three, roughly 9 seconds per combination, 1,000 combinations is about 150 minutes of model time for a full warming run, against about 90 minutes for 600. That is an extra hour for each complete run, and the run has to be repeated whenever the question set changes. During development the set changes often, so this is not only a cost the machine pays overnight; it is a cost I pay while working. I noted both of these as things to watch rather than reasons not to try. The point of the larger set was to find out what it actually gives, and that requires running it. There is also a question the larger set was meant to answer that has nothing to do with coverage. The matrix search rewritten on 28 July has never been measured, and its whole advantage is supposed to grow with the number of stored entries. A cache of 600 is small enough that the difference might not be visible. A cache of 1,000 is closer to a size where it should be. One thing I did not change was the priority weighting. The new templates and entities were added to the existing ordering rather than given their own, so the most likely questions are still generated first and the new ones take their place among them. This keeps the two versions comparable: if the larger set behaves differently, it is because it is larger, not because it is ordered differently. Next step: run the expanded set and see what the extra 400 combinations are worth in practice.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 05.08.2026
Yapılan Uygulama:	Evaluating the Expanded Set and Returning to 600	
Today I ran the warming process on the expanded 1,000 combination set and looked at what it produced. The larger set did not give an additional benefit that justified it. What it did do was make the warming run noticeably longer, in line with yesterday's arithmetic. The number of combinations sets the number of model calls almost exactly, so a set two thirds larger takes about two thirds longer to prepare, and there is no way around that: each combination is one question the model has not answered before. So I returned the configuration to 30 templates across 20 entities, 600 combinations. This is the size the system kept for the rest of the project. The change was easy to undo, which is worth noting as a matter of how the system is built. The question set is data, not code. Widening it and narrowing it again meant editing a list, not rewriting the warming process, and nothing else in the system had to know that the number had changed. If templates and entities had been written into the code instead, a day of work would have become several. It is worth being clear about why more prepared answers is not automatically better, because it is easy to assume it is. A prepared answer only has value if somebody actually asks that question. Every combination in the set costs the same amount of model time to prepare, whether it is a question users ask constantly or one nobody ever asks. When the set is small it holds the most likely questions, because those are the ones I wrote first and the priority weighting generates first. Widening it adds progressively less likely questions, so the cost per combination stays flat while the expected value of each new combination falls. At some point the additional combinations are paying full price for something that will probably never be used. So there is a size where widening stops being worth it, and this measurement put it below 1,000 for this system rather than above it. Reverting a change I had spent a day building is not a comfortable thing to write down, but it is a real result. The question "is a bigger prepared set better?" had an answer, the answer was no at this scale, and I now know that instead of assuming it. It is worth being precise about what this does and does not show. It does not show that 600 is the right size, only that 1,000 was not better than 600 here. Finding the best size would need a proper test against real user questions, which I do not have and will not have during this project. What I have is one comparison at one scale, and it points down rather than up.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 06.08.2026
Yapılan Uygulama:	Reviewing the State of the System Before the Next Stage	
Before moving on to the next stage, I went over the current setup to understand how it performs as a whole, rather than continuing to add to it. Four weeks of building one piece at a time makes it easy to lose track of what the whole thing is, and I had not written it down anywhere in full. At this point the system consists of the following, all of it built over the previous four weeks: - Two local servers, one generating answers on port 8000 and one producing embedding vectors on port 8001, both running through llama.cpp with no GPU and no internet access. - A cache class storing questions, answers and their vectors in a local SQLite database, with an eviction policy that removes the least recently used entry when the cache is full. - Three separate checks before a match is accepted: the similarity score against the 0.80 threshold, an entity check that rejects matches naming different subjects, and a topic check that rejects matches asking about different aspects of the same subject. - A lookup path that searches the cache with the raw question first and only pays for the standardisation model call if that finds nothing. - A retrieval path that splits documents into overlapping pieces, finds the closest ones to a question, and builds a prompt from them. - A warming process that prepares 600 combinations in advance during idle time, in priority order, reusing the live search path so it cannot disagree with it about what counts as a match. - A similarity search implemented as a single matrix operation rather than a loop over rows. Setting this out in one place was useful for seeing what has not been done rather than what has. The clearest gap is that the matrix rewrite from 28 July has never actually been measured. I rewrote it, I verified it produces identical results to the loop version, and then I moved on without ever checking whether it is faster or by how much. It is reasonable to assume a matrix operation beats a Python loop, but assuming is not measuring. At the moment the only honest thing I could say about that change is that it did not break anything. The second gap is related. Almost every number I have about this system is a single measurement taken once. That was fine while I was building, because I only needed to know whether something worked. It is not enough for a report, where a number is a claim. Writing the list also showed how much of it is protection rather than function. Three of the eight items exist only to stop the cache returning a wrong answer, and each was added after a wrong answer actually appeared. None of them was designed in advance. That is worth remembering when I plan the rest of the project: the parts I would not have thought to build are the ones the system needed most.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 07.08.2026
Yapılan Uygulama:	Weekly Meeting and Agreeing the Next Stage	
I had my weekly meeting with Prof. Wu and went through the week: the thread settings that produced no clear gain, the expansion to 1,000 combinations and the return to 600, and the state of the system as it stands. His feedback was to focus next specifically on vectorized, matrix based similarity, to make the matching both faster and more consistent when comparing similar questions. This lined up with the gap I had identified yesterday, and it covers more than it might sound. There are two separate pieces of work in it. The first is measurement. The matrix rewrite exists and is known to produce identical results, but its speed has never been measured against the loop version it replaced. Without that, it is a change I believe in rather than a result I can report. Measuring it properly means comparing the two implementations across a range of cache sizes, not only the current one, because the whole argument for the matrix approach is that its advantage grows with size. The second is the word "consistent", which points at something I skipped on 28 July. I decided then not to add an explicit normalisation step inside the matrix calculation, because the stored vectors are already close enough to unit length that the difference would not change any decision. That reasoning is about the size of the error, not about whether there is one. If a similarity score is going to be reported as a number and used to explain the system's behaviour, it should be a true cosine similarity rather than something that is almost one. So the next stage is to finish that change properly and then measure the result. We also talked about what happens after that. The system is now complete enough that adding more parts to it is not obviously the right move, and Prof. Wu's feedback points the same way: finish and measure what exists rather than build more. This is the first week where the work ahead is about proving the system rather than extending it. I sent my weekly progress report at the end of the day. Writing it was a good check on the week, because there was less to put in it than usual: two things tried that did not work, and one thing built and then taken back out. That is a fair description of the week and I wrote it that way, rather than making the report look busier than the work was. A weekly report that always shows progress stops being information. It also would not have survived the meeting, since Prof. Wu asks about the details.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 10.08.2026
Yapılan Uygulama:	Making the Similarity Calculation a True Cosine Similarity	
Following Friday's meeting I started on the change I had deliberately skipped on 28 July. Cosine similarity measures the angle between two vectors, ignoring how long they are. The formula divides the dot product of the two vectors by the product of their lengths. When both vectors already have length exactly 1, the division does nothing and the dot product alone is the cosine. That is the shortcut I was relying on: my vectors are normalised when they are stored, so they are unit length, so the dot product is the answer. The problem is that "already unit length" is an assumption, not a guarantee. Floating point arithmetic does not produce exact values, so a vector I normalised is unit length to within a very small error rather than exactly. That error is far too small to change a hit into a miss, which is why I skipped this in July. But the score is not only used for decisions. It is printed, recorded, and reported as a percentage in this report and in the ones I send to Prof. Wu. A number presented as a cosine similarity should be a cosine similarity. So today I added the normalisation explicitly inside the batch similarity function, where the matrix multiplication happens, instead of relying on the vectors having been normalised somewhere else earlier. The two places are different in an important way: normalising at storage time is a promise made by one part of the code and trusted by another, while normalising in the calculation makes the calculation correct on its own terms whatever it is given. I then tested that nothing changed, which was the part that mattered. I compared scores before and after on near unit vectors and found the difference negligible, as expected, and confirmed that no existing hit or miss decision changed. Writing "I made a change and nothing happened" sounds like a wasted day, so it is worth saying why it was not. There are two different things I could be doing. One is making the system behave better, and this change does not do that. The other is making the reported numbers mean exactly what they claim, and this change does. Before today, every similarity percentage in my reports was approximately a cosine similarity, and the size of the approximation depended on how well normalised the vectors happened to be. After today it is a cosine similarity because the function computes one, not because the inputs happened to arrive in the right shape. The difference does not show up in any result. It shows up in whether I can defend the number when someone asks what it is. There is a practical reason to do this now rather than later. The similarity score appears in the console output I keep as evidence, and that output goes into the reports. Changing how the score is computed after collecting evidence would mean the older output and the newer output were produced by different formulas, and I would have to explain which is which. Making the change before the measurement work starts avoids that. Next step: measure the matrix implementation against the loop it replaced.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 11.08.2026
Yapılan Uygulama:	Benchmarking Matrix Similarity Against the Loop Method	
Today I measured the matrix based search against the loop based one it replaced on 28 July. Until now I had only verified that the two produce identical results; I had never checked whether the new one is actually faster. The important decision was to measure across a range of cache sizes rather than only the current one. The argument for the matrix approach is not that it is faster in general, it is that its advantage grows with the number of stored entries, because the loop pays Python's per-iteration overhead once for every row while the matrix version does the whole thing in one operation regardless of row count. A single measurement at my current size would show a number but would not show that behaviour, and the behaviour is the reason for the change. So I ran both implementations at sizes from 50 up to 20,000 stored entries. The results confirmed the expected shape. At my current cache size of 600 entries, the matrix method was about 26 times faster. Across the full range of sizes tested, the speed-up was roughly 15 to 26 times. The reason to run the comparison rather than only the new version is that "faster" needs something to be faster than. I still had the loop implementation, so I could run both against the same data on the same machine in the same session. A number measured today against a number remembered from three weeks ago would not be a comparison, because too much else has changed since then. Two things about these numbers are worth recording honestly. The first is that the advantage is smallest at the smallest sizes, which is exactly what the reasoning predicts: with only 50 entries there is not much loop overhead to remove. That the measurement agrees with the prediction is a good sign that I am measuring what I think I am. The second is that 20,000 entries is far beyond anything this project uses. My cache holds 600. I tested up to 20,000 to show the trend, not because the system will get there. Presenting a speed-up measured at 20,000 entries as a result about this system would be misleading, so the number that describes this system is the 26 times at 600, and the range is context for how it behaves as it grows. This also settles the question from 6 August. The rewrite was not just harmless, it was worth doing, and now I can say by how much instead of assuming. One more thing the range shows is where the change stops mattering. At 50 entries the two methods are close enough that the choice between them would not be worth making. The reason to use the matrix version is not that it is faster today; it is that the loop version gets worse as the cache fills and this one does not. Next step: the measurement gave a total time for a search, but not a breakdown of where that time goes. Look at that next.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 12.08.2026
Yapılan Uygulama:	Finding That the Matrix Multiplication Was Not the Bottleneck	
Yesterday's benchmark measured how long a search takes. Today I broke that time down into its parts, and found that I had been optimising the wrong thing. A search is not only a calculation. Before any similarity can be computed, the stored vectors have to be read out of the SQLite database, converted back from the binary form they are stored in, and assembled into a matrix. Only then does the matrix multiplication run. I had been treating the multiplication as the cost of a search, because that is the part I rewrote and the part I was thinking about. The measurements showed that fetching the rows from the database and rebuilding the matrix accounted for 80 to 90 percent of the total time of a search. The matrix multiplication itself, the part I spent 28 July rewriting and yesterday benchmarking, was the small remainder. This does not mean the rewrite was pointless. The 26 times figure from yesterday is real, and the loop version paid the same database cost on top of a much slower calculation. What it means is that the rewrite removed most of the cost of one part of the search, and that part turned out not to be the expensive part. There is a much larger saving available that I had not looked at because I had not measured. The waste is structural rather than accidental. The stored vectors change only when something is written to the cache, which happens on a miss. But the matrix is rebuilt from the database on every single search, including the many searches that find nothing new and change nothing. The system throws away the assembled matrix after each search and builds an identical one on the next. Seeing this stated plainly makes the fix obvious: keep the matrix in memory and rebuild it only when the stored data actually changes. That is a cache in front of a cache, which sounds excessive until you look at the ratio between how often the data is read and how often it is written. Every question causes a read. Only a miss causes a write. As the cache fills, misses become rarer, so the proportion of wasted rebuilds grows over time rather than staying constant. I also want to record the pattern, because this is the second time. On 27 July I assumed a delay came from the model server and spent two days there before measurement pointed at my own connection code. Here I assumed the cost of a search was the calculation and rewrote it, before measurement pointed at the database read. Both times the assumption was reasonable. What was wrong was acting on it without checking first. The difference this time is that I measured before spending days on the next step rather than after. Next step: implement the in-memory matrix.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 13.08.2026
Yapılan Uygulama:	Adding an In-Memory Matrix Cache	
Today I implemented the fix for yesterday's finding: the assembled matrix is now kept in memory and reused between searches instead of being rebuilt from the database every time. The whole difficulty of this kind of change is not building it, it is making sure the kept copy never disagrees with the database. A stale matrix is worse than a slow one: the system would search a version of the cache that no longer exists, and it would do so quickly and without any error, so nothing would tell me it was wrong. Every way the stored data can change has to invalidate the copy. There are three such ways in this system, and I handled all of them. Saving a new entry after a cache miss adds a row. Evicting the least recently used entry when the cache is full removes one. Clearing the cache removes everything. Each of these now marks the in-memory matrix as no longer valid. Finding all three meant going through every place that writes to the database rather than only the obvious one, because a single missed path would produce exactly the silent failure I was trying to avoid. I chose to rebuild lazily rather than immediately. When the matrix is invalidated it is simply discarded, and the next search that needs it builds it again. The alternative, rebuilding straight away on every write, would do work at a moment when nothing is asking for the result, and several writes in a row would rebuild several times for one eventual read. I verified this with regression tests rather than by running it once and seeing a fast answer. A speed improvement is easy to observe and proves nothing about correctness. The failure mode here is a wrong answer, not a slow one, so the tests check that results after a write are what they should be, not just that they arrive quickly. The result is an additional 5 to 10 times speed-up for repeated searches. That qualifier matters. The first search after any write still pays the full rebuild cost, so this change does nothing for a system that writes as often as it reads. It helps exactly the pattern this system has, where a user asks many questions in a session and most of them are hits that change nothing. The prepared cache makes this more true, not less: the more questions are already answered, the higher the proportion of searches that read without writing. Combined with yesterday's finding, the picture for the search path is now complete. The calculation was made faster by the matrix rewrite, and the larger cost sitting behind it was removed by not repeating it. What is left of a cache hit is small enough that it no longer shows up next to anything else in the system, which is the point at which further work on this path would be wasted effort.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 14.08.2026
Yapılan Uygulama:	Deciding to Move the Project to an Electrical Engineering Field	
Prof. Wu was not available for our usual meeting today, so we spoke on the phone instead. The main subject was where the project goes next. The infrastructure has reached a point where the remaining items are refinements rather than missing pieces. The cache stores real semantic vectors, protects itself against two kinds of wrong match, searches by matrix operation, keeps that matrix in memory, fills itself in advance, and answers a repeated question in a fraction of a second. The retrieval path works. What the system does not have is a subject worth pointing it at. That was always a separate decision, because none of the work so far depends on the subject. The cache does not know what the questions are about, and the retrieval does not know what the documents contain. The field is decided by which documents are loaded and what the prompt says the assistant is, and both are content rather than architecture. We agreed to move the project to an electrical engineering field. There are two reasons for this. The first is my own: I am an electrical engineering student, this internship is part of my degree, and a system that retrieves and explains technical information is far more useful to me if the technical information is from my own field. The second is that the field suits the system. Electrical engineering runs on standards, technical manuals and equipment documentation, which are exactly the kind of source material a retrieval system is built for: they are long, they are written to be looked up rather than read through, and their content is specific and checkable rather than a matter of interpretation. We did not settle on the specific area today. Electrical engineering is wide, and not all of it suits a local retrieval system equally well. Prof. Wu asked me to look into the possibilities and come back with options, so that we choose deliberately rather than taking the first idea. I also thought about what makes an area suitable, since that is what I have to judge the options against. The system answers questions from documents, so the area needs documents that exist, that are public, and that contain specific answers rather than general discussion. It runs a small model on a CPU, so questions whose answers require long reasoning chains are a poor fit while questions whose answers are stated somewhere in a document are a good one. And I need to be able to check the answers myself, which means the area has to be one I understand well enough to know when the system is wrong. Next step: research candidate areas within electrical engineering, assess each against those requirements, and put the options to Prof. Wu with the weekly report on Monday.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 17.08.2026
Yapılan Uygulama:	Choosing the Electrical Engineering Area for the Project	
On Friday Prof. Wu and I agreed to move the project to an electrical engineering field, and he asked me to come back with options rather than a single suggestion. Today I sent him the weekly report together with five candidate areas. The five were: local AI for embedded systems documentation and technical support under limited computing resources; equipment fault and maintenance, using equipment data and fault records to suggest diagnostic steps; retrieval of information from electrical standards, manuals and technical documents; circuit and electronics troubleshooting; and power system decision support. I judged each against what this system can actually do rather than against how interesting it sounds. The system finds passages in documents and has a small model explain them. That rules out anything needing calculation, simulation or long reasoning chains, and favours anything where the answer already exists in writing and the difficulty is finding it. It also has to be an area where I can check the answers myself, because a system I cannot verify is one I cannot report on. Prof. Wu replied the same day and picked the third option: using retrieval to find and explain information from electrical standards, manuals and technical documents. He pointed out that the CPU-based local model with RAG and a semantic cache in front of it is a good fit for that kind of material. I agree with the choice, and the reason is the nature of the documents. Standards and manuals are written to be looked up rather than read through. They are long, organised into numbered sections, and full of specific checkable statements rather than discussion. Someone using one usually has a narrow question and has to find the paragraph that answers it. That is exactly the problem retrieval solves, and it is a real problem rather than one invented to suit the system. The second reason is verification. A standard states a rule with a number in it, so when the system answers I can open the document, find the paragraph and see whether the answer matches. In an area where a good answer is a matter of judgement, I could only form an impression. Here I can measure. That will matter a great deal at the evaluation stage. What does not change is the architecture. The cache, the two servers, the guards, the matrix search and the retrieval path all stay as they are. What changes is which documents are loaded and what the prompt says the assistant is. This is the separation I described in July, and this week is the first real test of whether it holds. If it does, six weeks of work carries over unchanged. If it does not, I will find out quickly, because the first real document will break something. One thing this decision does not settle is which documents to use. The area is chosen, but standards are not all freely available, and a system I cannot show to anyone because its source material is licensed would be a poor result for an internship report. That is the first thing to solve. Next step: confirm the direction with Prof. Wu and start loading an electrical engineering document into the retrieval pipeline.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 18.08.2026
Yapılan Uygulama:	First Electrical Engineering Document in the Retrieval Pipeline	
I confirmed the direction with Prof. Wu early this morning and started work the same day. With about a month of the internship left, the sooner the new field is actually running rather than agreed, the more time is left to measure it. The first thing the pipeline needed was a proper way to load a document. Until now, adding text to the retrieval store meant calling three functions in the right order by hand: split the text into pieces, turn each piece into a vector, save each one. That is fine while testing but wrong as a permanent arrangement, because the order and the settings have to be repeated correctly every time. So I wrote a single function that does all three, taking a document and its source name and handling the rest internally. The three functions underneath it did not change; I only wired them together. For the first test I wrote a short synthetic manual myself rather than using a real standard. It covers two rules for an industrial electrical installation: which breaker curve type a motor branch circuit needs depending on the motor rating, and how to size the conductor for that circuit. Writing it myself meant I knew exactly what the correct answers were, which is what I needed at this stage. A real document would have tested the pipeline and my understanding of the document at the same time, and if something went wrong I would not know which one was at fault. The retrieval half worked, and it worked on the first attempt. Questions about the manual pulled the right pieces from the right source, and the pieces contained the rules the questions were about. The conductor sizing question was answered correctly, including the percentage the rule specifies. This is the answer to the question I raised yesterday about whether the architecture carries over: nothing in the cache or the retrieval had to be changed to work on electrical engineering text instead of the previous material. The motor breaker question was not. For a 7 kW motor the model answered with the wrong curve type, even though the rule that decides it was sitting in the text given to the model. This is a different kind of failure from anything I have had so far. Every previous wrong answer came from the cache returning the wrong entry, which is a matching problem. Here the matching was correct, the retrieval was correct, and the model still read the rule and got it wrong. I decided not to chase this today. I have one occurrence, no measurement, and no way yet to tell whether it happens every time or once in five. Fixing something I cannot measure would mean changing settings until the answer looks right, which is guessing. This belongs in the evaluation stage, where I can run it repeatedly and see the pattern. Next step: load a real electrical engineering document. The synthetic manual has done its job, and the point of the project is documents I did not write.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Company Visit	Tarih: 19.08.2026
Yapılan Uygulama:	Technical Visit to Minth Group, Ningbo	
Today was a technical visit rather than a development day. Together with the two other interns from our office and the teacher assigned to us by the university, I visited Minth Group in Ningbo, a technology company that mainly produces parts for electric vehicles. The visit began with a presentation of the company, its products and its main areas of work. We were then shown around the laboratories and workshops. I should note that the company does not share details of its equipment, and mobile phones were collected into sealed cases at the entrance so that no photographs could be taken. For that reason I cannot record specific makes, models or test setups here, and there are no photographs of this day in the appendix. What I can describe is the general impression. The scale of a real production and testing environment is difficult to picture from a classroom, and what struck me most was how much of the floor space goes to measurement and quality checking rather than to production itself. After the tour we held a meeting with nine people in the room: two human resources staff, an electrical engineer, an aerospace engineer and a project manager from the company, and the three of us with our supervising teacher. They explained the company's goals, how their teams are organised and how a project moves through them. It was useful to hear an electrical engineer describe day to day work in an industrial setting, because it is a very different environment from the research office I have been working in for the past six weeks. We then had lunch together in a separate part of the company canteen. The conversation continued in a more relaxed form and covered both academic and cultural subjects. I talked about Turkish food and customs, and heard about theirs. This part of the day was not technical, but it was a real part of working abroad, and it is one of the things I will take away from this internship. We returned to the office in the afternoon. Prof. Kingsom Chow, who is responsible for the office, gathered everyone working there, about eight to ten students, in his room. He first asked each of us a few questions about our own area of work, then spoke for around an hour and a half about his own career and how he came to found the company. After a few more conversations the day ended at about half past five. Looking at the day as part of the internship rather than as a break from it, the value was in seeing where work like mine eventually goes. I spend my days on a retrieval system for technical documents. A company like this is where those documents are actually used, by engineers who need one specific rule from a long manual while standing next to the equipment it applies to. That is easy to lose sight of while measuring similarity scores. Next step: return to the real-document work planned for this week, one day later than intended.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 20.08.2026
Yapılan Uygulama:	Loading a Real Standard and Fixing Two Retrieval Problem	
Today I loaded the first real electrical engineering document into the system: a United States federal regulation covering wiring design and protection. I chose it for three reasons. It is public domain, so there is no licensing problem in using it. It contains concrete numeric rules, which are checkable rather than open to interpretation. And it is a genuine working document of the kind the final system is meant to serve. I also changed how documents are supplied. The synthetic manual from Tuesday was written inside the source code, which was acceptable for something I wrote myself but not for real material. The regulation is loaded from a text file in its own folder, which is how real standards and manuals will be added from now on. The load failed immediately. The embedding server rejected it, because a piece of text that my splitter counted as 500 words came to 802 tokens, above the server's limit of 512 in one batch. The cause is that my splitter counts words while the model counts tokens, and the ratio is not fixed. In ordinary prose a word is roughly one token. In a technical regulation it is not: section numbers, units and bracketed references each break into several tokens. A setting that was safe on the test material was unsafe here, and nothing warned me until the server refused the request. I fixed it for this document by loading it with a piece size of 120 words and an overlap of 25, which keeps it well under the limit. I deliberately did not change the splitter itself. It works for the material it was written for, and changing it under pressure without measuring would risk breaking what already works. I noted it to fix properly later if it happens on other documents. With the document loaded, a second problem appeared. A question with two numeric parts to its answer came back with only one of them. The retrieval was set to fetch the two closest pieces, and the piece holding the second number was ranked third, just outside that window. Raising the setting to four pieces brought it in, at a similarity score of 0.5027, and the model then answered both parts correctly. This is worth noting as a type of failure. Nothing reported an error. The answer looked complete and was internally consistent; it was simply missing a fact that the system had never been shown. Without knowing the correct answer in advance, I would have accepted it. Both of today's problems came from the same source: settings that were reasonable on the material I had and wrong on the material I now have. Neither was a mistake in the code. The architecture carried over to the new field without changes, as I hoped on Monday, but the settings did not, and those are two different claims. This is the real cost of moving a working system to a new kind of input, and it is worth writing down rather than passing over. Next step: connect the cache to the retrieval path. They are still two separate routes, and the retrieval function is not called during normal use.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 21.08.2026
Yapılan Uygulama:	Connecting the Cache to Retrieval and Limiting Runaway Answers	
Until today the cache and the retrieval path were two separate things. A question went through the cache, and on a miss it went straight to the model with no document context. The retrieval function existed and worked, but nothing in the normal path called it. Today I connected them, which was the main remaining item for the current stage. The change itself is small. The cache miss branch now calls the retrieval-backed answer function instead of asking the model directly. Everything else stays as it was: the fast path, the standardised lookup, the guards and the saving are untouched. A question now takes one of two routes. If the cache has it, the answer comes back with no model call. If not, the system finds the relevant document pieces, has the model answer from them, and stores the result. During testing I hit the worst failure of the project so far. One question took 282 seconds and produced nonsense: the model lost track of what it was doing and collapsed into repeating text, inventing content that was not in any document. It had never done this before. I have a plausible explanation. The retrieval store now holds two separate documents in one pool, and nothing tells the search to stay inside the one a question belongs to. The short manual I wrote on Tuesday has three pieces; the regulation loaded on Thursday has thirty-two. So a question about the manual can easily pull in regulation text that has nothing to do with it, and a small model given unrelated context alongside the right context is a known way to get this kind of collapse. But one occurrence is not evidence, so I did not treat it as settled. What I did instead was fix the consequence rather than the cause. There was no limit on how much text the model could produce in one answer, so a failure like this could run indefinitely. I added a cap of 500 tokens. This does not prevent a bad answer; it makes one cheap instead of costing five minutes. That seemed right while the cause was still uncertain. I then reran the same test. It did not happen again: the same question returned a clean answer in 12.80 seconds. So it appears to be a rare event rather than a reliable one, which is also why I could not have investigated it properly today. Something that happens once in an unknown number of attempts cannot be studied until I can run the same question many times and count, and I have no way to do that yet. At the weekly meeting Prof. Wu asked for a diagram of the system as it now stands. I drew the flow from a user question through the cache to either an immediate answer or retrieval and the model, and sent it with the weekly report. Drawing it after today's change was useful, because the diagram now has one path through it rather than two separate ones that happened to live in the same program. Next step: start measuring. Response times, retrieval speed, cache hit rate and whether the answers are actually correct.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 24.08.2026
Yapılan Uygulama:	Pipeline Verification and System Prompt Correction	
Friday's work connected the cache to retrieval, but I had only tested one question before the meeting. Today I checked it properly from end to end. The behaviour is what it should be. A question the system has not seen misses the cache, goes through retrieval, is answered from the document pieces, and is stored. The same question asked again in different words comes back from the cache without the model being called at all. That second part is the one worth confirming rather than assuming, because a stored answer that is never found again is just a slow system with a full database. Then I found something I should have caught a week ago. Every electrical engineering answer produced so far, including all of last week's testing, had been generated under the system prompt written for the previous field. The prompt tells the model what kind of assistant it is and what subject it answers about, and I had changed the documents, the questions and the retrieval without ever changing that. The model was being told it was one kind of assistant while being handed electrical engineering text. I replaced it with an electrical engineering version, keeping the same structure and tone and only changing the scope. I also added a line I did not have before: answer only from the information provided, and say so when the information does not cover the question. A system that retrieves from documents should be willing to say the documents do not contain the answer, and until now nothing had told it that. This raises a question about last week's results that I want to record honestly. The wrong motor breaker answer from Tuesday was produced under the old prompt. So was every answer I looked at on Wednesday and Thursday. I do not think the prompt caused that failure, but I cannot rule it out, and it means last week's observations are not a clean baseline. I finished the day by writing down where the project stands against the plan I set when the field changed. Preserving the existing structure is done. Building retrieval for electrical engineering documents is done. Adapting the cache is partly done: the cache and retrieval are now one path and the prompt is corrected, but the entity and topic vocabularies are still the ones written for the previous field. They are unused rather than wrong, since none of their words appear in electrical engineering text, but leaving them is a decision I will have to explain. Measuring and evaluating has not started, and it is now the main remaining work. There is one thing I want to note about how this was found. I did not find it by reading the code, and nothing in the output pointed at it. I found it while listing what had and had not been done against the plan. Going through a checklist is a slow and unexciting way to work, but it catches the things that never announce themselves, and a wrong system prompt does not raise an error. Next step: build the measurement harness. I have no numbers for this system at all.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 25.08.2026												
Yapılan Uygulama:	Building the Measurement Harness													
Everything I know about this system's speed comes from watching numbers scroll past in the console. That is enough while building and useless for a report, so today I built something that collects measurements instead of printing them. Three changes. First, the timings are now stored rather than only displayed, in a place any part of the code can write to without changing what any function returns. I deliberately avoided changing function signatures, because a measurement system that requires rewriting the thing it measures is a good way to introduce bugs into working code. Second, the retrieval path used to be timed as one block. That block contains three very different things: turning the question into a vector, searching the documents, and the model generating the answer. Measuring them together gives the total and says nothing about where it goes. That is exactly the mistake behind the matrix search work: I spent a day making one part faster before knowing what share of the total it was. They are now timed separately. Third, I wrote a runner that takes a list of questions, sends each through the system, prints a summary, and appends every result as a row to a file. The appending matters. Results accumulate across days instead of disappearing when the console scrolls, so I can compare a run from next week against one from today. The question set was designed rather than collected. Each subject starts with a base question, followed by rewordings of it. This means a cache hit has to be earned by matching a paraphrase rather than an identical string, which is the behaviour I actually want to measure. I also included one question from outside the field, to check that the system declines instead of inventing something. The first run gave the first real numbers for this system: <table><tr><td>Measurement</td><td>Value</td></tr><tr><td>Cache hit, mean</td><td>0.277 s</td></tr><tr><td>Cache miss, mean</td><td>10.113 s</td></tr><tr><td>Speed-up</td><td>about 37x</td></tr><tr><td>Document retrieval</td><td>0.0010 s</td></tr><tr><td>Model generating the answer</td><td>9.255 s</td></tr></table> The last two lines are the result. Retrieval takes a thousandth of a second and the model takes about ninety percent of the total. Everything I have built around the search — the matrix rewrite, the in-memory matrix, all of it — operates on a part of the system that was already close to free. That is not an argument against the cache. It is the argument for it, stated precisely. The cache is not valuable because searching it is fast. It is valuable because a hit means the model is never called. Next step: run the guards against electrical engineering questions. They were written for a different field and have never been tested on this one.			Measurement	Value	Cache hit, mean	0.277 s	Cache miss, mean	10.113 s	Speed-up	about 37x	Document retrieval	0.0010 s	Model generating the answer	9.255 s
Measurement	Value													
Cache hit, mean	0.277 s													
Cache miss, mean	10.113 s													
Speed-up	about 37x													
Document retrieval	0.0010 s													
Model generating the answer	9.255 s													
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU												

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 26.08.2026
Yapılan Uygulama:	Two Cache Bugs Found by Measurement	
Today the benchmark did what benchmarks are for: it found two bugs I had no idea were there. The first was the entity check. I had assumed it was inactive here, since it was built to compare country names and there are none in electrical engineering questions. It was not inactive. It was rejecting correct matches. The check decides what counts as a name using capital letters, and that rule misfires on this material in three ways: words that begin a sentence, like "For", are treated as names; contractions such as "What's" are not recognised either; and the pattern pulls a lone capital letter out of a token like "15A", so a unit becomes a name on its own. Two questions that should have matched produced disagreeing lists of nonsense names, and a match at 63 to 70 percent similarity was rejected because of it. I made the smallest fix that addresses the causes: strip trailing contractions, skip single letters, and extend the list of ordinary words that are never names. Before running it I predicted what the hit rate should become if my explanation was right. It went from 30 percent to 40 percent, exactly as predicted. Predicting first is worth the extra minute: a fix that improves the number for a different reason than I think is one I do not actually understand. The second bug I went looking for. In the previous field, two questions could differ only by which country they named, and the entity check existed for that. The equivalent here is a number: a question about a 15 A circuit and one about a 30 A circuit are nearly identical as text and have different correct answers. Nothing was checking that. I wrote a trap question to find out whether the problem was real before writing any code to fix it. It was. The 30 A question matched the stored 15 A entry at 58.13 percent similarity and returned the wrong figure — 12 amperes, where the correct answer is 24. Worse, it missed on the fast path and only matched after the standardisation step reworded it, which means that step can blur exactly the detail that distinguishes the two questions. So I added a third gate: extract the numbers from each question with their units, and reject a match if the asked value is not among the values the stored answer covers. It is deliberately cautious — a number only counts when a known unit follows it, so section and regulation numbers are ignored. After the fix, the hit rate fell from 41.7 percent to 33.3 percent. That is an improvement, not a regression: the hit that disappeared was returning a wrong answer. It is also a concrete reason not to report hit rate on its own, since the same measurement moved in both directions today for opposite reasons — up when a correct match was restored, down when a wrong one was removed. Both bugs have the same shape. A rule that was correct for the previous field is not correct here, and neither announced itself. One was rejecting good matches, the other accepting a bad one, and I would have found neither by reading the code. Next step: measure whether the answers are actually right. Everything so far is speed, and today showed the system can return a wrong answer instantly and confidently.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 27.08.2026
Yapılan Uygulama:	Measuring Whether the Answers Are Correct	
Everything I have measured so far is speed. A system can be fast and wrong, and yesterday showed that mine can be wrong in the worst way: instantly, from the cache, with no sign that anything went wrong. So today I added correctness to the benchmark. The difficulty is defining correct without reading every answer by hand. Reading them by hand does not scale: with repeated runs there will be hundreds, and I would grade the later ones more loosely than the first without noticing. I settled on storing, with each question, the phrases its answer must contain, grouped so that alternative wordings of the same fact count as one requirement. For the trap questions I also store phrases that must not appear, which is how the wrong cached value is caught coming back. The runner now grades every answer against this and reports accuracy alongside the timings. It counts one thing separately: wrong answers that came from the cache. A wrong answer from the model is a mistake. A wrong answer from the cache is worse, because it arrives instantly, costs nothing, and will be returned again to every rewording of the same question. I checked the grader against answers I already knew were wrong, rather than trusting it because it compiled. It correctly marks the motor breaker answer as wrong, and it catches the 30 A trap. Its limitation is that string matching cannot tell whether an answer contradicts itself, so a self-contradictory answer containing the right phrase would pass. Because of that, the full text of every answer is kept in the results file for manual review. The second addition was repeated runs. The model does not give the same answer to the same question every time, and its speed varies from run to run for reasons outside the code. A single run therefore measures the run, not the system. The runner can now repeat the whole set several times, clearing the cache between repeats so each starts cold, and report per question how many times out of how many it was correct. That last part changes what the results mean. "Three of five correct" is a completely different finding from always right or always wrong, and a single run reports one of the latter two by accident. A question that fails intermittently is telling me something about the model's consistency, and a question that fails every time is telling me something about the system. Until today I could not tell those apart. I now have a benchmark that measures speed, correctness and consistency, and writes all three to a file that accumulates. That is the tool I needed before any of the remaining questions can be answered properly, including last week's motor breaker failure, which I deferred precisely because I had no way to tell a systematic error from an occasional one. Next step: add a second real document, and stop questions about one document from pulling text out of another.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 28.08.2026
Yapılan Uygulama:	A Second Document, Source Separation and Diagnostic Tools	
Today I added a second real standard, from the same public domain family as the first, covering general requirements including working space clearances and the marking of equipment. The retrieval store now holds 65 pieces of text across three sources: this new standard, the wiring regulation loaded last Thursday, and the short manual I wrote myself. That made last Friday's problem impossible to leave alone. The mixing risk already existed with two documents, and I suspect it caused the runaway answer. A third makes it worse rather than creating it, and each further document will make it worse again, so this had to be fixed before adding any more. Retrieval now works in two stages. It first finds the single best matching piece across everything, then searches only within that piece's source document. This way the caller does not have to know which document a question belongs to, which a user never would. The question's own best match decides. I validated it offline first, by reconstructing the exact similarity scores from the real failure and checking that the old behaviour reproduces the mixing while the new one excludes the unrelated piece. I also checked the awkward cases: an empty store, a single document, and an explicitly named source. The rest of the day went to two diagnostic tools, and these changed how I work more than the retrieval fix did. The first answers a question I could not previously answer: when an answer is wrong, is it because the right text was never retrieved, or because it was retrieved and the model misread it? These need completely different fixes and look identical from the outside. The tool takes a question and a string from the correct answer, and reports where the piece containing that string ranks. It calls no model, so it is fast and gives the same result every time. The second is a controlled comparison. It asks the same question repeatedly under different settings, bypassing the cache so every attempt actually runs, and reports how many times each setting produced a correct answer. Yesterday I could observe that a change seemed to help. This tells me whether it helped or whether I saw noise. I also wrote a small separate script that reads the accumulated results file and produces a comparison table across runs. Individual runs scroll past; the table is what a report needs, and building it now rather than in the final week means the results file is being written in a shape I can actually use. The general point is that this week moved the project from building to measuring, and the two need different tools. Until Tuesday I had no numbers at all. Since then every change has been driven by a measurement rather than a guess, and two of them were things I would never have found by reading the code. The work has not become easier, but I now know whether I am making it better. Next step: the remaining weakness is table data. A question needing one row of a table still fails, and I now know why: that piece scores 0.43 and ranks thirteenth, so it never reaches the model.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 31.08.2026
Yapılan Uygulama:	Writing the August Monthly Progress Report	
The end of the month means a monthly report, in the same six section structure as the one I wrote in July. This one covers the change of field, the retrieval work on real documents, and the measurement stage. I wrote a first draft and then cut it heavily. The draft ran to nineteen pages, and reading it back I could see that a good part of that length was there to look thorough rather than to say anything. A reader who gives up halfway through nineteen pages has learned less than one who finishes twelve. So I cut anything that was included only to fill space, and rewrote the sentences that dressed a simple fact in complicated language. The final version is twelve pages with five figures. I then went through the whole thing again, checking every number against the output it came from rather than against my memory of it. That pass found six errors, and I want to record them because finding them is the point of doing it. One was a calculation: I had worked out a model inference time by hand instead of taking it from the results file, and got it wrong. One was a claim I could not support — I had written that two runs used the same code on different days, which I believed but could not show. One was a set of answers I had recorded during testing and presented as if they were console output. One was a chunk count in a timeline figure that did not match the actual count. One was a formatting fault where a shared list counter made the Next Steps section start at item four. And one was more subtle: I had changed a grading label partway through collecting results, which made one configuration's accuracy read 0.7 points lower than it actually was. None of these would have been caught by rereading the text. Rereading catches sentences that sound wrong. It cannot catch a number that reads perfectly well and is simply not what the program produced. They were only found by going back to the source of each one, which is slower and is the only method that works. The last one is worth keeping in mind for the final report. It was not a mistake in the report at all; it was a mistake in how the data was collected, and the report repeated it faithfully. Checking a document against its sources catches errors in the document. Catching this kind means checking whether the sources themselves were produced consistently, which is a different and harder question. Next step: the plan for the next two weeks is to make document ingestion sustainable — chunk sizes that are calculated rather than guessed, and adding a document by copying a file rather than editing code. Then more real documents, and measure again.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 01.09.2026
Yapılan Uygulama:	Automated Token Density and Chunk Sizing	
The chunk size of 120 words has been a magic number since 20 August. I found it by trial and error after a load failed, it works, and I do not know why 120 rather than 100 or 150. That is acceptable for one document and not acceptable as a method, because the next document will have different density and I will be guessing again. There is also a practical reason to fix this now. The plan for the next two weeks includes adding more real documents, and if every new document needs a session of trial and error before it will load, adding documents becomes expensive enough that I will add fewer of them. So today I measured the density instead of guessing it. The results were more interesting than I expected. Measured per character, this material is ordinary: about 4.65 characters to a token, which is close to normal English. Measured per word it is not: about 1.6 tokens to a word, where normal prose is around 1.3. The reason is that technical terms are long. A regulation is not dense because it is packed with symbols; it is dense because its words are. That explains why splitting by word count was risky and it also gives a safe fallback. Word count is the unreliable measure here. Character count is not. I built the change in two parts, without touching the existing splitter at all. The first counts tokens properly, by asking the embedding server how many tokens a piece of text contains. If the server cannot be reached it falls back to the character based estimate, deliberately set to overestimate. An overestimate produces chunks that are smaller than necessary, which costs a little accuracy. An underestimate produces chunks the server refuses or silently truncates, which is much worse. The second measures each document's own token to word ratio and works out a chunk size from a budget of 200 tokens, which sits comfortably under the embedding model's practical window. Loading a document now takes no size argument by default. It checks the longest chunk it produced against the real limit and shrinks up to three times if needed. I kept the option of passing an explicit size, because controlled experiments need to fix that variable rather than let it be calculated. The result is a good sign. Given the same document, the automatic method chose 125 words, against the 120 I had found by hand. The two agree closely, which means the method reproduces a human judgement I already trust while being able to make that judgement for documents I have not seen. What I have not done is check whether this changes the results. The chunk sizes it produces are close to the ones I was using, but close is not identical, and different chunk boundaries mean different retrieval. That has to be measured rather than assumed. Next step: make adding a document a matter of copying a file rather than editing code.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 02.09.2026												
Yapılan Uygulama:	Folder-Based Loading and a Run That Came Out Worse													
Adding a document still meant writing a block of code for it, and there were three such blocks. Today I replaced them with one function that reads every text file in the documents folder and loads any that is not already stored. Adding a document is now copying a file. I moved the manual I wrote in August out of the source code and into that folder, so all three documents now arrive by the same route. Testing this against a stub database covered four cases: loading an empty store, running again and skipping what is there, detecting a newly added file, and a missing folder. The test also showed me a mistake of my own. When I moved the manual into a file I added a note at the top saying it is not a real standard. That note is part of the file, so it was being split into chunks and could reach the model as if it were content. I removed it. The file name and the report both make its status clear, and text meant for a reader has no business inside the retrieval store. Because chunk boundaries had changed, I labelled this run separately so its results are never averaged with the earlier ones. I also made a prediction before running, which is a habit I picked up last week. The manual should now be two chunks instead of three, with the two competing rules together and in document order. That is the exact situation I had blamed for the long standing wrong answer on the motor question, so I expected it to be fixed. The run came out worse than the hand tuned baseline: <table><tr><td></td><td>Manual, 120 words</td><td>Automatic</td></tr><tr><td>Accuracy</td><td>74.7%</td><td>68.9%</td></tr><tr><td>Cache miss time</td><td>8.0 s</td><td>13.2 s</td></tr><tr><td>Inconsistent questions</td><td>1</td><td>6</td></tr></table> My prediction was wrong, and wrong in a useful way. The manual did not become two chunks; it became one, because the calculated size of 138 words is larger than the whole 133 word document. So the model received all three rules together, complete and in order, and still gave the wrong answer, three times out of three. That result kills two explanations at once. It cannot be missing context, because nothing was missing. It cannot be chunk ordering, because there was only one chunk. Both of these were hypotheses I had been carrying since August, and a single run has removed them. Next step: find out why the accuracy dropped. I have a suspicion about the chunk size calculation.				Manual, 120 words	Automatic	Accuracy	74.7%	68.9%	Cache miss time	8.0 s	13.2 s	Inconsistent questions	1	6
	Manual, 120 words	Automatic												
Accuracy	74.7%	68.9%												
Cache miss time	8.0 s	13.2 s												
Inconsistent questions	1	6												
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU												

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 03.09.2026
Yapılan Uygulama:	Investigating the Regression and Designing a Controlled Test	
Today I tried three explanations for yesterday's drop. None of them survived, which is a slower kind of progress than finding the answer but is still progress. The first was my own suspicion. The chunk size calculation measures a document's density from its first 300 words, and one of the documents starts with plain prose and keeps its tables at the end. If the sample were lighter than the document as a whole, the calculated size would come out too large. I checked, and the opposite is true: the opening is denser than the average, so the estimate errs on the safe side. The second was prompt size. Measured in tokens rather than words the chunks barely moved, from about 192 to about 200. A prompt four percent larger does not explain a cache miss taking seventy one percent longer. The third came from looking at which questions changed. Four got worse and two got better, and the interesting pair are the two table questions: one fell from always correct to a third of the time, the other rose from never correct to a third. They moved in opposite directions. My guess was that changing chunk boundaries had shifted whether a table row sat with its heading. I checked both, under both configurations, and in every case the row and its heading are in the same chunk. That explanation fails too. So I cannot account for the drop with the data I have, and I would rather write that down than pick the least bad guess. There is also a reason the two runs cannot settle it. They differ in more than one thing: the chunk boundaries changed, but the model was also running slower across the whole session, up from about 7.2 to about 12.3 seconds, and the two runs used different numbers of repeats. Comparing them tells me something changed, not what. One possibility I could rule out entirely. If the chat server were started with a small context window, a larger prompt might be silently cut off, which would explain the drop and several older oddities as well. It is started with a window of 4096 tokens and 12 threads. Eight retrieved pieces come to roughly 1600 tokens with the prompt and question, plus a 500 token cap on the answer: about 2100 against 4096. There is no silent truncation. That check produced something else worth recording. Both servers run at once and share the same processor during a benchmark, which is the most likely reason the model's speed swings between runs on unchanged code. So the timing difference should not be attributed to my change, and I will treat speed and accuracy separately from now on. Next step: a proper test that changes one thing. Keep the automatic method, set the token budget to 175 so it reproduces the configuration already known to work, and use the same number of repeats as the baseline so the two are directly comparable.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 04.09.2026
Yapılan Uygulama:	Confirming the Chunking Method and Fixing Table Retrieval	
The controlled run settled it, and gave the best result of the project so far: 78.7% accuracy against the previous best of 74.7%. The comparison is clean, which is the point of having designed it that way. Across fifteen questions and five repeats, exactly one question changed: the one asking about something no document covers, which went from correct once in five to four times in five. Everything else is identical, including the three questions that are always wrong. The reason makes sense. The automatic method gave the densest document 112 words per chunk and the others 121, because it measures each separately. The hand tuned setting forced 120 on all three. Smaller chunks mean less context, and less context makes the system better at saying it does not know. That matches a finding from late August, when adding context made this same question start inventing an answer instead of declining. It also explains Wednesday's drop. Same code, one parameter different: a budget of 200 tokens gives 68.9% and 175 gives 78.7%. So the chunk size was the cause after all, and the budget is now a single number with a measured reason behind it rather than a guess. To be fair, the repeat counts differed between those two runs, three against five, but a ten point gap and a fall from six inconsistent questions to one point clearly in one direction. I spent the rest of the day on the table question that has never worked. The diagnostic showed its answer ranked thirteenth at 0.43 similarity, so the model never sees it. Today I found out why. The chunk holding that table row also holds unrelated prose before it, about preventing contact with live parts. The table content is diluted by text that has nothing to do with it, so the chunk does not look much like a question about the table. A second chunk holds the row without its heading, which is no better. The fix is to stop tables and prose from being mixed. Tables are now separated out and split on their own, with the heading repeated on each chunk they produce. Prose is chunked exactly as before, by the existing splitter, which I again did not modify. The only new rule is that a prose window may not run across a table. I checked four things offline before running anything. The problem row is now in a pure table chunk of 140 tokens, carrying a heading whose wording overlaps the question closely. A second table became its own chunk of 78 tokens holding both of its rows with the heading. The manual is unchanged at two chunks. And a document with no tables chunks identically to before, so prose retrieval should not regress. Having been wrong on Wednesday, I am stating this as an expectation rather than a prediction. I expect the table question to improve, and the manual questions not to change since it has no tables. The check that matters most is whether the twelve questions that currently work still work, because the largest risk in a new chunking method is breaking what already functions. I had my weekly meeting with Prof. Wu and sent the progress report.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 07.09.2026												
Yapılan Uygulama:	Measuring the Table-Aware Chunking Change													
On Friday I separated tables from ordinary text when splitting a document, and said what I expected rather than predicting it, having been wrong the week before. Today I ran it. The result is the best so far: 85.3% accuracy, against 78.7% for the previous configuration. Fifteen questions, five repeats each, same question set and same settings as before, so the two runs are directly comparable. The question this change was aimed at is now answered correctly. It asks for a value that lives in one row of a table, and it had never once worked. Before the change, the passage containing that row also contained unrelated prose about preventing contact with live parts, so the passage as a whole did not resemble a question about clearances. It scored 0.43 and ranked thirteenth, well outside the eight passages the system actually reads. Now the row sits in a passage made only of table content, with its heading repeated at the top, and the heading's wording overlaps the question closely. The check I cared about more was the one for damage. A new splitting method is exactly the kind of change that fixes one thing and quietly breaks three others, and the twelve questions that already worked are worth more than the one that did not. They all still work. A document with no tables is split into exactly the same passages as before, which is the property I built the change around. So the accuracy progression across the whole series is now measured rather than assumed: <table><tr><td>Configuration</td><td>Accuracy</td></tr><tr><td>Top 4 passages, hand-tuned size</td><td>72.6%</td></tr><tr><td>Top 8 passages, hand-tuned size</td><td>74.7%</td></tr><tr><td>Automatic size, 200-token budget</td><td>68.9%</td></tr><tr><td>Automatic size, 175-token budget</td><td>78.7%</td></tr><tr><td>Table-aware splitting</td><td>85.3%</td></tr></table> I am keeping the 68.9% row in that table rather than removing it. It was my first attempt at automatic sizing and it made the system clearly worse. Only one number differed between that row and the one below it, which is what turned "chunk size matters" from an opinion into a result. A table that only shows the steps that went upward would be a less honest table and a less useful one. Two known failures remain and neither is affected by today's change. One question about motor breaker curves is wrong in every repeat, and has been since the Electrical Engineering work began. One question whose answer is in no document refuses correctly four times in five and invents an answer on the fifth. Next step: there is a bug I have been aware of and have not fixed. If I edit a document, the old passages stay in the database.			Configuration	Accuracy	Top 4 passages, hand-tuned size	72.6%	Top 8 passages, hand-tuned size	74.7%	Automatic size, 200-token budget	68.9%	Automatic size, 175-token budget	78.7%	Table-aware splitting	85.3%
Configuration	Accuracy													
Top 4 passages, hand-tuned size	72.6%													
Top 8 passages, hand-tuned size	74.7%													
Automatic size, 200-token budget	68.9%													
Automatic size, 175-token budget	78.7%													
Table-aware splitting	85.3%													
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU												

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 08.09.2026
Yapılan Uygulama:	Detecting Changed Documents	
Adding a document became a file copy last week. Editing one was still broken, and today I fixed it. This is the last real bug in the system rather than a limitation of it. The problem had two halves. The loader skips a file that is already in the database, which is what makes repeated runs cheap, but it decided that by name alone. So an edited file was skipped entirely and its old passages stayed. Worse, any answer already cached from the old version kept being returned instantly and confidently, with nothing to indicate it was built from text that no longer exists. A stale document is bad; a stale cached answer is worse, because it is fast and carries no warning. The fix stores a fingerprint of each file's contents alongside its name. On loading, the system compares the file against its stored fingerprint. Unchanged, it skips as before. Changed, it removes that document's passages and the cached answers that were built from it, then loads the new version. The second half of that is the part that needed care. Removing the passages is straightforward. Removing the right cached answers is not, because an answer may have been assembled from more than one document. I already store which sources each cached answer came from, so the rule is to remove an answer if the edited document is among its sources and to leave everything else alone. An answer built purely from an untouched document keeps working, which matters because clearing the whole cache on every edit would make editing a document expensive enough that I would avoid doing it. I tested this on a temporary database rather than my working one. Four cases: an unchanged file is skipped, an edited file is reloaded, the cached answers derived from it are removed including one assembled from several sources, and answers from other documents survive untouched. One more case needed handling. My own database was created before any of this existed, so it holds passages but no record of which file version they came from. Without a guard, the loader would see no fingerprint, treat every document as new, and load a second copy of everything alongside the first. So a database with passages but no version records now replaces them rather than adding to them. This is a small feature and it took most of the day, almost all of it on the cases where it could go wrong rather than on the case where it works. That ratio seems to be normal for anything that deletes data. A feature that adds something can be checked by looking at the result. A feature that removes something has to be checked by looking at everything it did not remove. Next step: the interactive mode. I have not looked at it since the field changed, and I need it working for demonstrating the system.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 09.09.2026
Yapılan Uygulama:	Rebuilding the Interactive Assistant and Testing It by Hand	
The interactive mode is how anyone other than me would actually use this system, and I had not opened it since July. Everything since then has been driven by the benchmark, which is a good way to measure a system and a poor way to notice that it is unusable. Today I found it in worse condition than I expected. It still introduced itself as an assistant for the previous field, which is embarrassing but harmless. The real problem is that it never loaded the documents at all. On my own machine it appeared to work, because my database already held passages from benchmark runs. On a fresh database it would have answered nothing, and I would have discovered that in front of someone. I rebuilt it. It now loads the document folder on startup and reports how many passages it holds, so an empty corpus is visible immediately instead of looking like a system with nothing to say. For every answer it shows where the answer came from: either that it was a cache hit, with the similarity score and the time, or which document it read. That last part matters more than it sounds. The whole subject of this project is the cache, and a system that answers quickly without saying why is indistinguishable from a system that is simply fast. Then I used it myself, typing questions the way a person would rather than running the benchmark, and that found something the benchmark cannot. I asked two questions that are related to a question already in the cache but ask something different: one about conductor sizing only, and one asking for a section number. Both matched the cached answer about breaker curves, at around 60% similarity, and returned it. The similarity threshold is 0.55, so nothing malfunctioned. The system did exactly what it is configured to do. The benchmark cannot see this because of how I built the question set. Its rewordings genuinely ask the same thing, and they sit in the same 61 to 74 percent band as these mismatches. The band does not separate them. Raising the threshold to around 0.65 would reject today's bad matches, but it would also reject a legitimate rewording in my own benchmark that matches at 60.99 percent. So this is a real trade-off and not an obvious fix. I am recording it rather than changing it, because with the code freezing in two days there is no time to measure the change properly, and an unmeasured change to the threshold would undo the one thing this project can claim: that its settings have reasons behind them. Next step: the motor breaker question. I now have document change detection, which gives me a way to test it that I did not have before.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 10.09.2026
Yapılan Uygulama:	Explaining the Failure That Has Been Open Since August	
One question has been wrong in every repeat since the Electrical Engineering work began, and two of my explanations for it have already been disproved. Today I settled it. The day started badly. The change detection I built on Tuesday existed in my working copy but had never reached the machine I run the benchmark on, so that database had no version records at all. I found this while setting up today's test, verified the feature end to end on a temporary database, and only then continued. Noting it here because a feature that exists in one place and not the other is a failure of delivery rather than of code, and it would have quietly invalidated everything I did afterwards. Then the test. The manual I wrote myself contains a conductor sizing rule at 125 percent. I edited the file to say 130 percent, confirmed the change reached the database and that both stored passages contained the new value, and asked the question. The system answered 125 percent. That number is the real value from the National Electrical Code, which the model would know from its training. The document in front of it said 130. So the model was not reading the document; it was answering from what it already believes. I checked that retrieval itself was not at fault, because the whole conclusion rests on that. I asked for the section number covering conductor sizing in that manual. The answer, Section 4.3.5, is a number I invented; it exists nowhere except in my file, and no model could produce it from training. It answered correctly, reading two passages. So retrieval works, grounding works, and document edits do reach the model. The failure happens specifically where the document contradicts something the model already believes about electrical practice. The breaker question has the same shape: my manual requires a Type D curve above 5 kW, which runs against common practice, and the model keeps answering Type C. The last question was whether an instruction could override it. I built a small experiment that retrieves once, then varies only the wording and placement of the instruction across four variants: the current prompt, an override line in the system message, the same line next to the retrieved text, and both. Placement usually matters more than wording, so varying it was the point. All four variants were wrong in all five trials. Twenty out of twenty. This closes the question as a limitation of the small local model, not a bug in my pipeline and not a prompt problem. It also costs me one benchmark question: the 3 kW case is answered correctly every time, but the model's own prior also says Type C there, so it cannot tell grounding apart from guessing. I would rather lose a question than keep one that looks like evidence and is not. It took three weeks and two wrong explanations to get here, and the thing that finally worked was inventing a fact no model could know and asking for it.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 11.09.2026																				
Yapılan Uygulama:	Freezing the Code and Running the Final Measurement																					
Today the code is frozen, on the date planned two weeks ago, which leaves the final week entirely for measurement and writing. First I restored the benchmark. Yesterday's experiment had been wired into the main block with an early exit so it would run alone, and that exit had to come out or today's run would have measured nothing. I commented the experiment out rather than deleting it, with its result in the comment, so the finding survives without the code path. Then the final run, on exactly the code being submitted. Fifteen questions, five repeats: <table><tr><th>Metric</th><th>Value</th></tr><tr><td>Accuracy</td><td>84.0% (63/75)</td></tr><tr><td>Cache hit</td><td>0.202 s</td></tr><tr><td>Cache miss</td><td>7.925 s</td></tr><tr><td>Speed-up</td><td>about 39x</td></tr><tr><td>Semantic search</td><td>0.0054 s</td></tr><tr><td>Document retrieval</td><td>0.0010 s</td></tr><tr><td>Model inference</td><td>90% of a miss</td></tr><tr><td>Cache hit rate</td><td>26.7%</td></tr><tr><td>Wrong answers served from cache</td><td>5</td></tr></table> This 84.0% is the figure I will report, even though it is lower than Monday's 85.3%. It is the honest one, for two reasons. It was produced by the frozen code in one controlled run. And Monday's label turned out to hold two runs, not one: the run I reviewed scored 85.3%, a second accumulated under the same label scored 82.7%, and one of the two may have been taken while the manual was temporarily edited for yesterday's test. That label is not a clean single run and I will not quote it as one. The final run also produced one new observation. A question that had been solid became inconsistent, correct in three of five repeats, and its two failures took 30.9 and 38.3 seconds against 9.1, 13.7 and 9.8 for the three successes. The two groups do not overlap at all. This is the same family as the 282 second runaway in August and the small text corruptions I have seen since: when generation runs unusually long, the answer degrades. It also means the 84.0% figure carries some dependence on machine conditions, which I will state rather than hide. I am not chasing it. The code is frozen, and a fix I cannot measure properly in the time left is worth less than an accurate description of the limit. I sent the progress report covering the last two weeks and had my meeting with Prof. Wu. What remains is writing, plus one decision to record rather than implement: the topic vocabulary and the cache pre-warming from the previous field stay unadapted, and the report will explain why instead of leaving them looking unfinished.			Metric	Value	Accuracy	84.0% (63/75)	Cache hit	0.202 s	Cache miss	7.925 s	Speed-up	about 39x	Semantic search	0.0054 s	Document retrieval	0.0010 s	Model inference	90% of a miss	Cache hit rate	26.7%	Wrong answers served from cache	5
Metric	Value																					
Accuracy	84.0% (63/75)																					
Cache hit	0.202 s																					
Cache miss	7.925 s																					
Speed-up	about 39x																					
Semantic search	0.0054 s																					
Document retrieval	0.0010 s																					
Model inference	90% of a miss																					
Cache hit rate	26.7%																					
Wrong answers served from cache	5																					
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU																				

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 14.09.2026
Yapılan Uygulama:	Choosing How to Benchmark the System	
The code has been frozen since Thursday, so this week is measurement and writing. The first question is what to measure, and that had been open since late August. I had asked Prof. Wu whether the benchmark should be about speed or accuracy, and whether it should use our own documents or a standard dataset. He left the choice to me, so I decided it today and wrote down the reasoning, because an arbitrary choice of benchmark is worth very little. What I have so far is an end-to-end measurement: ask a question, see whether the final answer is right. That is the honest headline number, but it mixes two components together. If an answer is wrong, the measurement cannot say whether the retrieval failed to find the passage or the model misread a passage it was given. The work of the past three weeks was almost entirely on retrieval, so a measurement that cannot see retrieval separately cannot show whether that work was worth doing. So the main measurement is a retrieval benchmark, using the two standard metrics for this: recall at k, which asks whether the passage containing the answer is among the k passages the system reads, and mean reciprocal rank, which asks how high up it was. These are what the well known retrieval benchmarks report, they need no download and no model calls, and they are deterministic, which matters more than it sounds. Every number I have reported so far comes from a model whose output varies between runs. These do not vary at all. I also considered a standard dataset. The natural one is the electrical engineering section of a public multiple-choice benchmark, which would measure what the model knows on its own, without documents. That is directly relevant, because my central finding from last week is that the model follows its own knowledge when a document contradicts it. Measuring the strength of that prior knowledge would put a number on it. But it comes with its own corpus and moves away from an offline setup built around specific documents, so I treated it as optional and secondary. I then built the retrieval benchmark as a separate program rather than adding to the pipeline. The pipeline is frozen, and a benchmark that modifies what it measures is not a benchmark. This one only reads: it rebuilds the corpus under each chunking configuration in its own temporary store, measures, and touches nothing belonging to the frozen system. The comparison is across three configurations: the hand-tuned 120 word chunks from August, the token-aware sizing without table handling, and the frozen system with both. That is the same sequence of decisions the report describes, measured on the part of the system those decisions actually changed. One thing I am not doing is inventing a new question set for this. The benchmark uses the same fifteen questions the end-to-end measurements use, so the two can be read against each other. A retrieval result on a different set of questions would be a separate finding rather than an explanation of the one I already have. Next step: prepare the ground truth and run it.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 15.09.2026																				
Yapılan Uygulama:	Running the Retrieval Benchmark																					
Today I ran the retrieval benchmark, and it gave the clearest result of the project. First the ground truth. For each question I need to know which passage contains the answer, which cannot be guessed from the question. I wrote a short distinctive string from the correct passage for each one, and checked that every string appears exactly once in the whole corpus. If a string appeared twice, the measurement would be scoring the wrong passage without saying so. Two of my fifteen questions have no answer in any document, so they cannot have a correct passage; I excluded them and report them separately rather than counting them as failures. That leaves thirteen. I also left out precision at k deliberately, and wrote down why. With exactly one correct passage per question, precision can never exceed one divided by k, so it would measure the size of k rather than the quality of the retrieval. Reporting it would look thorough and mean nothing. <table><tr><th>Configuration</th><th>R@1</th><th>R@4</th><th>R@12</th><th>MRR</th></tr><tr><td>Hand-tuned 120 words</td><td>85%</td><td>92%</td><td>92%</td><td>0.890</td></tr><tr><td>Token-aware, prose only</td><td>62%</td><td>92%</td><td>100%</td><td>0.779</td></tr><tr><td>Frozen system</td><td>85%</td><td>100%</td><td>100%</td><td>0.910</td></tr></table> The frozen system never misses a question at any depth from four onward, and ranks answers higher on average than either earlier configuration. The clearest single result is one passage, the one holding the working-space clearance table. Across the three configurations its rank for the question that needs it goes from fourteenth, to eighth, to second. The system reads eight passages, so at rank fourteen it was never seen, at rank eight it was barely inside, and at rank two it is read every time. That matches exactly what the end-to-end measurement showed: that question went from wrong in every repeat to correct in every repeat. Two independent measurements, one deterministic and one not, describing the same change through different means. The middle row is worth keeping even though it is the worst of the three. It is the configuration I built first, and its recall at one is well below the hand-tuned baseline it was meant to improve on. With the accuracy drop I measured two weeks ago, that is a second independent sign that it was a step backwards. There is also a result I did not expect. The frozen system reaches complete recall by the fourth passage, which means reading eight is more than it needs. Reading four would make every cache miss faster and might improve the system's willingness to say it does not know, since I have measured twice that extra context makes refusal worse. I am recording this as future work rather than acting on it, because the code is frozen and an unmeasured change now would undo the discipline the rest of the project was built on. The optional dataset did not work out. The only full distribution I could find is a large archive on a university server, about five hours on this connection, and the mirror I found contains only humanities subjects. I left the download running in case it finishes, but the report does not depend on it.			Configuration	R@1	R@4	R@12	MRR	Hand-tuned 120 words	85%	92%	92%	0.890	Token-aware, prose only	62%	92%	100%	0.779	Frozen system	85%	100%	100%	0.910
Configuration	R@1	R@4	R@12	MRR																		
Hand-tuned 120 words	85%	92%	92%	0.890																		
Token-aware, prose only	62%	92%	100%	0.779																		
Frozen system	85%	100%	100%	0.910																		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU																				

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 16.09.2026
Yapılan Uygulama:	Checking the Statistics, Then Writing the Final Report	
Before writing anything I did one check I had been putting off, and it changed how the report is framed. All my accuracy figures come from fifteen questions repeated five times, which I have been reporting as seventy-five answers. Seventy-five sounds like a reasonable sample. It is not, because those answers are not independent: the same question repeated five times gives five correlated results, and treating them as seventy-five separate observations makes the numbers look far more precise than they are. Counting each question once instead, the frozen system's 84.0% carries a confidence interval of roughly 66 to 100 percent, and the hand-tuned baseline's 74.7% roughly 53 to 97 percent. Those two intervals overlap almost completely. So on this question set, I cannot claim that one configuration is more accurate than another. The difference I have been describing is real in the data I have, but thirteen answerable questions are not enough to prove it. This does not weaken the report as much as it first seemed. It changes what the report leads with. The deterministic measurements involve no sampling at all: a passage either ranks second or it ranks fourteenth, and running it again gives the same answer. Those results, and the per-question changes they explain, are the defensible claims. The aggregate percentages stay in, described as what I measured rather than offered as proof that one setting beats another. So the report leads with mechanism. A table row was buried in a passage full of unrelated prose, it was separated out, its rank moved from fourteenth to second, and the question that needs it went from never right to always right. That chain is checkable at every step and does not rest on a sample size. I then wrote the report, in the same six section structure as the monthly ones: objectives, background, related work, the solution, the experimental study, and the conclusion. Writing to a structure I did not choose is easier now than it was in July, mostly because I have more to put in it. Two decisions went in as decisions rather than omissions. The topic vocabulary and the cache pre-warming from the earlier field were never adapted to electrical engineering, and the report says so and explains why, instead of leaving them looking unfinished. They are not wrong, they are unused, and adapting them at the end without time to measure the result would have been worse than leaving them. I also included the configuration that made things worse, with its numbers. A report showing only the changes that helped would be a shorter and less honest account of how the system reached the state it is in. It is also the more useful account: that row is the reason the chunk size setting has a measured justification instead of being another number I picked. Next step: check every number in the report against the output it came from.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 17.09.2026
Yapılan Uygulama:	Verifying the Report and Delivering It	
Today I checked the report and delivered it. The checking took most of the day, which is what I expected after doing the same thing to the August report two weeks ago. The method is the one that worked then. I did not reread the text looking for mistakes, because rereading catches sentences that sound wrong and cannot catch a number that reads perfectly well and is simply not what the program produced. Instead I went through the report claim by claim, and for each number found the file or console output it came from. Anything I could not trace was either corrected or removed. That rule removed some sentences I liked. A claim that cannot be traced is not a claim, whatever I believe about it, and the difference between the two is invisible to a reader, which is exactly why the rule has to be mechanical rather than a matter of judgement. I paid particular attention to three things. The first is the figure I report for the system's accuracy. There are two candidates and the higher one is not usable: the label it was collected under accumulated two separate runs rather than one, and one of them may have been taken while a document was temporarily edited for an experiment. The number in the report is the lower one, from the single controlled run on the frozen code, and the report says why. The second is the language about the statistical result from yesterday. It is easy to write a sentence that technically admits a limitation while being phrased so that a quick reader takes the opposite meaning. I checked those sentences specifically, and wrote them so that the limitation is the plain reading rather than the careful one. The third is consistency between documents. The report, the daily pages of this internship report, and the certificate from the company all describe the same work, and a reader may well have more than one of them open. Numbers, dates and the description of the system have to agree across all of them. I delivered the report to Prof. Wu in the evening, which was the plan set two weeks ago when the code freeze was scheduled: freeze on the eleventh, measure and write in the final week, and deliver before the last day. Keeping the last day clear was deliberate. Something always takes longer than expected, and this way a delay would cost me a margin rather than the deadline. Looking at the whole eleven weeks, the part I did not anticipate is how much of the work turned out to be measurement rather than building. The system as it stands is not large. What took the time was finding out what it actually does, which twice turned out to be different from what I was sure it did. Next step: the final day. Signatures, the company certificate, and handover.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

Uygulamanın Yapıldığı Birim:	Research and Development	Tarih: 18.09.2026
Yapılan Uygulama:	Freez Final Day, Handover and Closing the Internship	
Today is the last day of the internship. The technical work finished on the eleventh with the code freeze, and the report was delivered yesterday evening, so today was about closing things properly rather than producing anything new. I went through the handover. The frozen code, the accumulated measurement file, the document folder and the reports are all in one place, so that the work can be picked up by someone else without me explaining it. I wrote down the two things that are not obvious from reading the code: which measurement labels are clean single runs and which are not, and why the components left over from the earlier field were not adapted. I also collected the documents I need for my university. The company issued the internship certificate covering the dates, the scope of the work and the conditions, and the daily pages of this report were signed and stamped. I said goodbye to Prof. Wu, who supervised the project from the first day and read a written report from me every week for eleven weeks, and to Prof. Kingsum Chow and the other students in the office.		
Öğrenci - Mustafa Emre ÇEKMiŞ	Onay	Firma Yetkilisi- Prof. Tony WU

